

Full Unit - Final Report

A report submitted in part fulfilment of the degree of
BSc (Hons) in Computer Science and Mathematics

Supervisor: Julien Lange



Department of Computer Science
Royal Holloway, University of London

April 12, 2024

Declaration

This report has been prepared on the basis of my own work. Where other published and unpublished source materials have been used, these have been acknowledged.

Word Count: 25655 words

Student Name: Ashara Dangallage don

Date of Submission: 12/04/2024

Signature: Ashara Dangallage don

1 Table of Contents

1	Table of Contents	2
2	Rationale	4
2.1	Abstract	4
2.2	Introduction	4
2.3	Objectives	5
2.3.1	Term 1	5
2.3.2	Term 2	6
2.3.3	Project Plan Timeline - Term 1	7
2.3.4	Revised Timeline - Term 1	7
2.3.5	Revised Timeline - Term 2	8
3	Literature Review and Background	8
3.1	History	8
3.1.1	Hadoop Timeline	9
3.2	Hadoop Ecosystem	9
3.3	Hardware	11
3.3.1	Cluster	11
3.3.2	DataNode	11
3.3.3	NameNode	12
3.3.4	JobTracker	12
3.3.5	Gateway and other Auxiliary services	12
3.3.6	Network	13
3.4	Technologies/Applications	13
3.5	Hadoop Competitors	14
3.5.1	Apache Spark	14
3.5.2	History	15
3.5.3	Differences	15
3.6	Advantages/Disadvantages of Hadoop	16
3.6.1	Advantages	16
3.6.2	Disadvantages	17
4	Contents and Knowledge	19
4.1	Software Engineering	19
4.2	Testing	20
4.2.1	Shell Script	20
4.2.2	Excel	20
4.3	Nuclear Decay	20
4.4	Financial Concepts	21
4.4.1	RSI	21
4.4.2	Price Change	22
4.4.3	Trading Range	22
4.4.4	Volume/VROC	22
4.5	Map and Reduce	23
4.5.1	Map & Reduce	23
4.5.2	Combine & Partition	24
4.6	Garbage Collection Time	24
4.7	Parallelism	24
5	Project Progression	25
5.1	Problem 1	25
5.1.1	Class Diagram WordCount.java	30
5.2	Problem 2	31
5.2.1	Class Diagram RegexSearch.java	34
5.3	FileInputNumberFormat	35

5.4	Problem 3	36
	5.4.1 Description	36
	5.4.2 Technical Decisions	37
	5.4.3 Dataset	37
	5.4.4 Code	37
	5.4.5 Parallelism	40
	5.4.6 Frequency Distribution	41
	5.4.7 FrequencyDistribution.py	42
	5.4.8 Interpretation	43
	5.4.9 Performance Analysis	44
5.5	Problem 4	46
	5.5.1 Description	46
	5.5.2 Dataset	46
	5.5.3 Parallelism	46
	5.5.4 Price Change & Trading Range	47
	5.5.5 Volume Rate of Change	48
	5.5.6 RSI	49
	5.5.7 Interpretation	50
	5.5.8 Python Files	52
	5.5.9 Performance Analysis	53
6	Critical analysis and Discussion	55
	6.1 Reflections	55
	6.2 Difficulties	56
	6.3 Successes	57
	6.4 Future Enhancements	58
7	Professional Issues	58
	7.1 Hadoop	58
	7.2 Big Data	59
	7.3 Final Year Project	60
8	Appendix	60
	8.1 Diary	60
	8.2 README file	64
9	Acronyms	65
10	Glossary	66
11	Bibliography	67

2 Rationale

2.1 Abstract

This project delves into leveraging Hadoop MapReduce and its associated technologies for performing data analysis on a large collection of text files and datasets. In order to utilise and gain knowledge about the capabilities of Hadoop, I aim to address four problems: word occurrence extraction, distributed grep functionality, basic statistical analysis and advanced data mining algorithm development. For the first problem, I will extract and calculate the number of word occurrences in a series of web pages, thus gaining a basic understanding of Hadoop. For the second problem, I will implement a distributed grep functionality such as searching for strings matching a given regular expression within a web page collection. The third problem involves statistical analysis on a dataset and examining mode, minimum, maximum, median values and visualising the shape of the frequency distribution. The fourth problem requires the extraction and analysis of certain features from a NASDAQ dataset in order to uncover insights into from the data.

2.2 Introduction

In the field of data analysis, technologies like MapReduce have become crucial for efficiently processing vast datasets and as such, I spent the entirety of term 1 attempting to gain a fundamental understanding of it due to my novelty with the subject. However, despite my initial unfamiliarity, I was able to successfully complete all four problems outlined in the final year project.

Due to my keen interest in data analysis and big data, as well as my aspirations for a career in related fields, I decided to select the topic "Massive scale data analytics with Hadoop" for my final year project. Throughout the project, I dedicated myself to gaining an understanding of Hadoop and its applications in data analysis. Indeed, the choice of the NASDAQ dataset for problem 4 was influenced by my professional goals. Moreover, through research and coding sessions, I gained insights into the functionality of MapReduce and its potential applications in data analysis. In the end, I am confident that the skills I developed during this project such as using MapReduce for programming problems and deriving insights from data analysis will benefit me for a possible future career in a data analysis related field.

During my first term, my focus was on resolving the first three problems. However, due to time constraints, I was only able to successfully address 2 of them. Problem 1 involved extracting and counting word occurrences from a collection of poetry related web pages whilst Problem 2 focused on implementing a distributed grep functionality to search for specific string patterns within a collection of philosophy related web pages.

During my second term, my primary focus was on addressing the tasks presented in problems 3 and 4. Problem 3 involved the analysis of statistical data from a nuclear decay dataset to determine metrics such as mean, mode, maximum etc. as well as visualising the frequency distribution of the data. Problem 4 involved analysing metrics from a NASDAQ dataset such as daily price changes, daily trading ranges, daily rate of change of volume etc. in order to uncover insights from the data. Additionally, despite my initial plans to modify parameters and cluster configurations, I decided not to pursue such plans since the changes were unnecessary due to the small size of the dataset and the minimal impact they would have had.

2.3 Objectives

The primary objective of this project is to gain an understanding of Hadoop and perform data analysis using MapReduce. To achieve this, the project involves addressing four problems: extract and count word occurrences, implement a distributed grep functionality, perform basic statistical analysis on a dataset and analyse certain features to uncover insights from a NASDAQ dataset containing trading information.

2.3.1 Term 1

In Term 1, my primary objective was to gain a good understanding of the foundations of Hadoop MapReduce and acquaint myself with its syntax in order to prepare myself for problem 4 in term 2. In my approach, I did extensive research on Hadoop (1) and its components such as MapReduce, through resources such as books (e.g. "Hadoop: The Definitive Guide")(2), web pages and online forums. Despite initial challenges in finding specific details, this research proved valuable for my understanding of the topic which was later enhanced through coding sessions.

In weeks 2 and 3-5, I aimed to address challenges in problem 1 and problem 2 in order to test my comprehension of Hadoop. However, in week 2, due to a misunderstanding, I used Hashmaps instead of fully utilising MapReduce's mapper and reducer classes, which ended up wasting me the whole week. Due to this reason, I started problem 1 during week 3 and it was nearly completed by week 4. In week 4, I began working on problem 2, nearly finishing it by the first week, but struggled with an issue regarding the calculation of the line number.

Indeed, by week 7, despite many attempts, the problem still persisted which further broadened the gap between my current and expected progress. Moreover, during one of my meetings with my supervisor, he suggested the use of two mappers to chain them together, but after multiple attempts, this method proved ineffective. Nevertheless, after a series of discussions with him, this approach proved to be unnecessary and once again there was a misunderstanding. It was only in week 8, after detailed discussions during one of my regular FYP meetings, that he noted that the input key represented by TextInputFormat of the mapper class actually corresponded to the line number/byte offset. By the end of the week, I managed to successfully complete problem 2.

Weeks 6-8 were initially designated for addressing problem 3 and week 9 was designated for code optimisation and visualisations with Apache Superset. However, due to a delay in completing problem 2 by week 8, only some limited time remained for tackling problem 3. Hence, after a couple of discussions with my supervisor, we decided it would be more feasible for me to tackle Problem 3 during term 2. Nonetheless, I initiated work on my interim report and concluded problem 2 by the end of week 8 and thus I managed to meet the minimum expectations outlined by my project plan for Term 1.

As the term approached its conclusion in weeks 10-11, my focus shifted towards preparing for the interim report. Indeed, I was able to submit a few drafts of my report to my supervisor and receive constant feedback which was beneficial in ensuring that my interim report was concise.

In the end, term 1 was intended to assist me in developing my knowledge of Hadoop and address problems 1, 2 and 3. Although I did not complete problem 3, I successfully managed to complete problem 1 and 2 and I did gain an understanding of Hadoop through all my efforts during this term.

2.3.2 Term 2

During Term 2, my main goal was to complete my final year project by addressing problems 3 and 4 and submitting my final report. However, I encountered some difficulties relating to problem 3 at the beginning, particularly in calculating the mode. To overcome this difficulty, I chose to calculate each statistic for problem 3 in individual java files and then later aggregate everything into one java file. Despite these initial challenges, I was able to successfully calculate all the necessary statistics for problem 3, determine various metrics for a stock for problem 4 and create graphs to visually represent data for specific metrics by the end of the term.

In Week 1, I began experimenting with the calculation of specific statistics for the dataset "NuclearDecay" and I successfully managed to determine the mean for the variable "massExcessUncertainty". However, I encountered challenges in calculating other statistics within the same java file, which ultimately led to a slowdown in progress. In week 2, I made some progress by successfully calculating the cumulative frequencies for each value in the "massExcessUncertainty" column, which was a step intended to later calculate the mode. Nevertheless, despite my efforts, calculating the mode proved to be more complex than what I anticipated.

During week 3, I encountered challenges with calculating the mode using only the mapper and reducer classes, as this method was essential to consolidate all Java files for each statistic into a single Java file due to the key-value format for the mapper class. In the following week, I explored any solutions for consolidating multiple calculations into a single output file and through research and discussions with peers, I learned about the option of calling `context.write()` multiple times and about the `cleanup()` method, which executes after all the map tasks are completed.

Over the course of the following three weeks, I dedicated my time to finishing the calculations for problem 3. In Week 5, I focused on calculating the minimum and maximum values for the variable "massExcessUncertainty" in separate Java files. By Week 6, I successfully calculated the median and mode individually and by the end of the week, I aggregated these calculations into a single Java file called "NuclearDecay.java". Finally, in Week 7, I merged the mode and median calculations into the main Java file "NuclearDecay.java," which enabled me to output all variables together in a single output file ("part-r-0000").

In Week 8, I developed a Java program to calculate the cumulative frequencies for values in the "massExcessUncertainty" column and used Python to visually represent the results on a graph. Indeed, for this task I reutilised code from a previous commit for the partial mode calculation and borrowed a snippet of code to adjust the bar interval/length in matplotlib, which is a library I had limited experience with. In the end, I successfully finished Problem 3 by the beginning of this week.

During Week 9, I successfully computed the daily price range and change for a stock (Agilent Technologies Inc.) from a NASDAQ dataset and generated graphs of the data using Python. In Week 10, I extended my analysis by computing the volume rate of change for the same stock and visualising the results with matplotlib. The calculations were relatively straightforward due to the simplicity of the equations and the similarity of the Python code needed to create the graphs. Moreover, during this week I started my work on my final project report, where I began with the literature review section.

During Week 11, I made significant progress on completing the code, which included the calculation of the final metric for problem 4, the RSI. While there were challenges with implementing the RSI calculation, after conducting online research I managed to overcome these obstacles. Additionally, I made progress on sections such as the Rationale, Professional Issues and Literature Review on my report. However, the final draft I submitted to my supervisor was incomplete, for which I received minimal feedback. In Week 12, I requested an extension and I focused my efforts on completing the report and making minor adjustments to the test shell script. Additionally, I made improvements to `RegexSearch.java` to allow for the input of a regex expression from the command line.

In the end, in term 2, my goals were to complete problems 3 and 4 as well as to finish my final report. While I successfully tackled problems 3 and 4, I deviated slightly from the original plan outlined in my project plan. Initially, I had planned to utilise apache superset for generating graphs, but after discussion with my supervisor and realising its limitations with only csv files, I decided to use the python library "matplotlib" instead. Additionally, although I was supposed to implement a data mining algorithm, I chose to manually calculate metrics after discussing it with my supervisor, mainly due to time constraints and simplicity. Furthermore, I had planned to optimise parameter settings in week 9, but the size of my dataset meant that any changes to configuration settings would have been negligible on Hadoop's performance.

2.3.3 Project Plan Timeline - Term 1

Term 1	
Week 1	Gain an understanding of the foundations of Hadoop and familiarise myself with its syntax.
Week 2	Address problem 1
Week 3-4	Address problem 2
Week 5	Familiarise myself with Apache Spark and practise some of its tools.
Week 6-8	Address problem 3
Week 9	Optimise code and visualise results for problem 3 on Apache Superset
Week 10-11	Preparation for interim report
Term 2	
Week 1	Research for an adequate data mining algorithm and ensure data integrity and completeness within the NASDAQ dataset
Week 2-3	Eliminate any duplicate data within the dataset and transfer the data to Hadoop HDFS.
Week 4-8	Begin data processing by utilising the adequate data mining algorithm. Employ Apache Spark to derive further insights and visualisations.
Week 9	Optimise code and visualise results for problem 4 on Apache Superset
Week 10-11	Preparation for final report and presentation

2.3.4 Revised Timeline - Term 1

Term 1	
Week 1	Research about MapReduce to prepare for problem 1
Week 2	Made a mistake by addressing problem 1 through Hashmaps.
Week 3	Worked on problem 1 and completed it for the most part
Week 4	started addressing problem 2 and covered most of its functionalities, Made slight modifications to problem 1
Week 5-6	Worked on problem 2 trying to find a solution for the line number problem
Week 7	Made a mistake by trying to chain 2 mappers for the line number problem
Week 8	Finished problem 2, started addressing interim report
Week 9	Worked on presentation and submitted it, Made progress of Interim Report
Week 10	Prepared for the presentation and submitted Interim Report

2.3.5 Revised Timeline - Term 2

Term 2	
Week 1	Calculated the mean for problem 1
Week 2	Made progress towards calculating the mode
Week 3	Was stuck on how to calculate the mode so I spent more time researching the subject
Week 4	Made research on how to output multiple calculations under one single output file and decided to calculate the statistics separately
Week 5-6	Calculated the minimum and the maximum for problem 3
Week 7	Calculated the mode and the median separately for problem 3
Week 8	Calculated the frequency distribution and visualised it for problem 3
Week 9	Calculated the daily price change and the daily trading range for problem 4
Week 10	Calculated the Volume Rate of Change for problem 4
Week 11	Finished calculating the RSI for problem 4 and started to work on my final year report
Week 12	Worked on a few sections on my Final Report
Week 13	Finished and Submitted my Report

3 Literature Review and Background

Hadoop revolutionised the way we handle and analyse large datasets by providing a scalable and distributed storage, which enabled organizations to efficiently manage, process and derive valuable insights from massive volumes of information.

3.1 History

The origins of Hadoop trace back to 1997 when Doug Cutting (3) (co-founder of Hadoop) initiated the Lucene project which was an open source full-text search library that later became an Apache project. Subsequently, Doug and Mike Cafarella collaborated on Apache Nutch which was a project designed to index web pages using the Lucene library. However, as they encountered problems related to scalability with the increasing number of web pages to index, they sought a system that was fault-tolerant and ensured data integrity. Later, they were inspired by Google's File System and developed the Nutch Distributed File System (NDFS) in 2004, which addressed the challenges they faced in scaling Apache Nutch. Moreover, they later integrated NDFS with MapReduce and Hadoop Common to create the Hadoop project in 2006, which they named it after Cutting's son's toy elephant. Recognizing Hadoop's potential, Yahoo! enlisted Doug to enhance their search performance, resulting in multiple companies operating 1000 Hadoop node clusters by 2007.

Nevertheless, challenges emerged with scalability limitations in NameNodes and DataNodes, which prompted Hadoop to undergo through multiple versions, each of which brought improvements in its quality up until now, only three versions have been released.

In fact, six years after its initial release, Hadoop version 1 introduced capabilities that enabled MapReduce distributed computing on the Hadoop Distributed File System (HDFS) for distributed storage. This release also included major performance enhancements and robust security support, alongside notable improvements in HBASE (open-source distributed big data store).

However, the version 2 release of Hadoop made significant improvements compared to its previous version as it introduced YARN, which is a general purpose resource manager and job scheduling component. HDFS high availability, HDFS federations and HDFS snapshots were also introduced in later version 2 releases. However, the most recent major release of Hadoop is version 3, which introduced HDFS erasure encoding, a new YARN Timeline service with new architecture, YARN opportunistic containers, YARN distributed scheduling, support for three name nodes and Intra-DataNode load balancers. In addition to these features, version 3 also includes performance improvements and bug fixes.

3.1.1 Hadoop Timeline

- 2003 • Research paper for Google File System released
- 2004 • Research paper for MapReduce released
- 2006 • Hadoop Nutch created
 - Hadoop created by decoupling NDFS and MapReduce from Nutch
 - Project named Hadoop
 - Cluster size at Yahoo! increases to 600
- 2007 • Hadoop released with HBase
- 2008 • First Hadoop summit
 - The Yahoo! cluster receives a daily data load of 10 TB on Hadoop
- 2009 • Hadoop is utilised for sorting petabytes of data
 - Hadoop core renamed Hadoop Common
 - HDFS becomes a separate subproject
 - MapReduce becomes a separate subproject
- 2010 • Apache Hive released
 - Apache Pig released
- 2011 • Apache Zookeeper released
 - Hadoop cluster at Yahoo consists of 42000 nodes analysing petabytes of data
- 2012 • Hadoop version 1.0 released
 - Hadoop YARN released
- 2013 • Hadoop version 2.0 released
- 2017 • Hadoop version 3.0 released

3.2 Hadoop Ecosystem

The Apache Hadoop Ecosystem (established in 2006) is a suite of open source tools facilitating storage, processing and analysis of large datasets. It is built upon the Hadoop Distributed File System (HDFS) and MapReduce and has evolved to include projects such as Hive, Pig, HBase, Spark, Kafka and Flink (offering high-level abstractions, NoSQL databases and real-time streaming). Overall, the ecosystem mainly consists of four elements:

1. Hadoop MapReduce is a distributed framework that efficiently processes large datasets in a fault-tolerant and scalable manner. It functions in two stages: the Map phase, where data is transformed into key-value pairs and the Reduce phase, where the processed data is aggregated

and combined. In MapReduce, the data is initially divided into blocks and stored in the Hadoop Distributed File System (HDFS) and during the map phase, the blocks are distributed to cluster nodes and processed in parallel. The resulting output (represented as key-value pairs) is then shuffled and sorted before the Reduce phase, where it is aggregated and combined to generate the final output.

2. The Hadoop Distributed File System (HDFS) was first developed in 2005 as a solution to scaling challenges in Apache Nutch and its precursor, Nutch Distributed File System (NDFS), was inspired by Google's File System. HDFS is specifically designed for handling massive datasets and it offers fault tolerance, scalability and high throughput by distributing data across commodity hardware. To ensure fault tolerance, files in HDFS are divided into blocks, typically 128/256 MB in size, that are then replicated across multiple nodes. Moreover, the architecture of HDFS follows a master-slave model, with the NameNode managing the metadata and the DataNodes storing the actual data.
3. Hadoop YARN revolutionised distributed computing by separating resource management from the Map-Reduce paradigm as it allows multiple processing engines to efficiently share and utilise resources in a Hadoop cluster. In fact, YARN was developed to address the limitations found in Hadoop 1.0, providing a more flexible and scalable architecture with a separate resource management layer. This architecture solves challenges related to job scheduling, resource utilisation and supports various processing frameworks apart from MapReduce. Ultimately, it consists of a global ResourceManager, per-node NodeManagers, an ApplicationMaster for each application and the utilisation of containers for resource management.
4. Hadoop Common is a set of utilities and code libraries that support various Hadoop modules. Indeed, it provides a standardised platform for distributed storage and the processing of large datasets. To begin with, Hadoop Common was created in 2006 to address problems related to web crawling and indexing. It includes the Hadoop Distributed File System (HDFS) for distributed storage, utility tools for MapReduce data processing and the Hadoop API in Java for application development. Additional components such as Hadoop Common Utilities and Hadoop Authentication improves its functionality by providing essential tools for system interaction and ensuring secure cluster access.

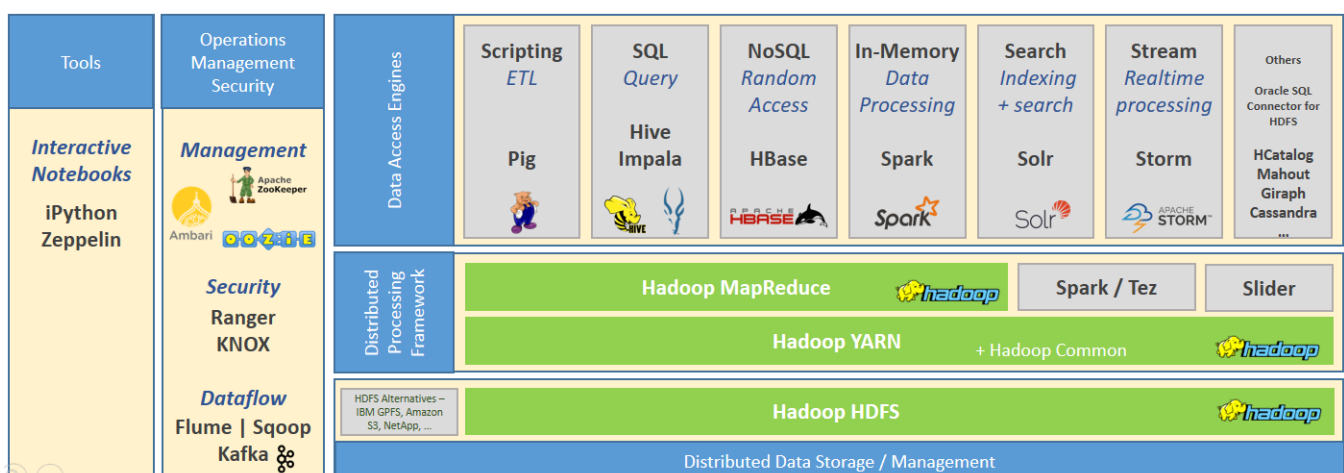


Diagram of Hadoop's Ecosystem

3.3 Hardware

Hadoop has become a fundamental technology that facilitated the storage, processing and analysis of large volumes of data in distributed computing environments. Hence, It is important to note that the effectiveness and performance of a Hadoop cluster is not limited to software alone; the hardware infrastructure also plays a critical role in achieving optimal operation and resource utilisation.

3.3.1 Cluster

The architecture of Hadoop comprises of interconnected master and slave nodes which utilises low-cost commodity hardware to ensure high availability. This design allows Hadoop clusters to achieve scalability by enabling the addition or removal of nodes to efficiently accommodate fluctuating data volume demands. Consequently, the hardware required for Hadoop must efficiently process small data portions on each node without extensive reliability and redundancy requirements.

Hadoop clusters are structured with master nodes such as the NameNode, the Secondary NameNode, the JobTracker etc., where each of which typically runs on separate machines to oversee operations across the distributed file system. Meanwhile, worker nodes (which mainly consists of virtual machines) manage data storage and processing tasks by running DataNode and TaskTracker services on commodity hardware as instructed by the master nodes.

The master nodes are responsible for overseeing important tasks such as data storage in HDFS and parallel computations through MapReduce. On the other hand, client nodes play a vital role in data transfer, job submission and retrieval within the cluster where they handle tasks such as data loading, result retrieval and submitting MapReduce jobs for processing, while also fetching the results upon completion. Working together, these components create a cohesive system where master nodes coordinate operations, worker nodes carry out storage/computation duties and client nodes facilitate data transfer and job management to ensure the smooth functioning of the Hadoop cluster.

3.3.2 DataNode

Within a Hadoop cluster, the DataNode serves as the primary storage component which manages the storage of HDFS data and the execution of MapReduce tasks. To improve data reliability, Hadoop utilises software-based redundancy measures by duplicating data blocks across various nodes to mitigate potential hardware failures. This approach improves fault tolerance and eliminates the need for conventional RAID setups in favour of more cost effective and efficient JBOD configurations.

In Hadoop clusters, DataNodes not only handle storage responsibilities but also execute MapReduce tasks. Indeed, by distributing tasks across multiple nodes and automatically retrying failed tasks on different nodes, Hadoop ensures consistent job performance even in the event of failures. Simultaneously, it also monitors node performance, promptly blacklisting nodes with high failure rates or slower response times in order to maintain overall cluster reliability.

Traditionally, Hadoop clusters have been built using servers with low storage density which enabled the scalability to petabytes with smaller hard drives. While hard drive capacity has improved, concerns about costs persist which results in a continued preference for large low density clusters. However, simply maximizing storage on each DataNode without sufficient CPU and RAM resources for efficient data processing is not effective. In response, many companies are now considering the development of more compact Hadoop clusters with increased storage and computational capabilities per server. These clusters not only address such challenges, but they also support workloads with heavy computational demands such as machine learning, exploratory analytics and similar tasks.

3.3.3 NameNode

The NameNode plays a critical role in maintaining the availability of the Hadoop Distributed File System (HDFS) by storing the filesystem metadata, which includes data on file blocks, their locations on DataNodes, available free blocks and suitable hosting servers. In addition, the NameNode is also responsible for handling heartbeat messages from all DataNodes in the cluster, which requires significant CPU resources to manage filesystem requests.

Initially, there was a concern regarding a single point of failure, which went against Hadoop's principles of fault tolerance. However, in Apache Hadoop 2.0.0, this issue was addressed by implementing the NameNode High Availability setup, though it required specific hardware different from that of DataNodes. Additionally, proper memory allocation for the NameNode is essential as it stores and enables quick access to metadata across various machines. Indeed, each HDFS block uses around 250 bytes of RAM on the NameNode, in addition to another 250 bytes necessary for files and directories. Despite this, the total memory usage increases significantly, particularly in large Hadoop clusters.

In order to maintain metadata consistency, the NameNode depends on two essential files: the editlog, which records all modifications to the HDFS and the fsimage, which holds a snapshot of the current filesystem state. If a server crash or restart was to occur these files allow the NameNode to recover a consistent filesystem state. Moreover, the Secondary NameNode plays a key role in this process by periodically merging changes from the editlog into the fsimage, which minimises recovery time.

3.3.4 JobTracker

In a Hadoop cluster, the JobTracker operates as a master server alongside the NameNode and Secondary NameNode, playing a critical role in managing job submissions. In fact, TaskTrackers located on each DataNode interact with the JobTracker to provide updates on job statuses and available processing resources.

Given their pivotal roles, ensuring reliable hardware for master nodes such as the NameNode and JobTracker is vital for sustaining cluster stability. Indeed, failure of the NameNode leads to a loss of access to HDFS data, while problems with the Secondary NameNode can cause delays in filesystem checkpointing. Additionally, a crash of the JobTracker can halt running MapReduce jobs and hinder the initiation of new ones. To prevent possible downtime, implementing strategies like RAID arrays for data volumes, redundant network/power supplies and high-quality enterprise grade components is crucial. This is particularly critical for the JobTracker, which has limited disk persistence compared to the NameNode.

3.3.5 Gateway and other Auxiliary services

Gateway servers serve as clients access points to the Hadoop cluster, which facilitates interaction with data in the HDFS while addressing network design and security obstacles. These servers are typically deployed (often hosting additional infrastructure components) outside the primary cluster subnet and user programs.

Moreover, gateway hardware requirements are less demanding compared to cluster nodes and they typically require 4-8 CPU cores and 16-24 GB of RAM for optimal performance. Additionally, gateways serve an important function in connecting users to the distributed computing environment, which provides a user-friendly interface for data access/manipulation while ensuring compatibility with the network architecture and security protocols of the cluster.

3.3.6 Network

In a Hadoop cluster, the network plays a crucial role alongside the CPU, disk and RAM in enabling communication between components. Both the HDFS and MapReduce rely significantly on network communication for tasks such as updating the NameNode, transferring data blocks and exchanging status messages. Indeed, In larger clusters, it is common to utilise lower-spec machines with fewer resources per node in order to meet capacity demands.

3.4 Technologies/Applications

Hadoop provides organizations and industries with the ability to manage and process large amounts of data efficiently. Due to its versatile functionality, Hadoop offers a wide range of applications spanning from healthcare to telecommunications, where it serves a critical role in reshaping data management practices and driving innovation. Some notable examples of these applications are:

- In healthcare (4), Hadoop's capabilities to address disease treatment, cost reduction, epidemic prediction and health metric tracking benefits the entire industry. In the case of genetic disease research, Hadoop successfully mapped the complete human genome (5), which consists of nearly three billion base pairs and this breakthrough holds the possibility of identifying cures for various diseases in the future. Moreover, Hadoop's functionalities are applicable in healthcare insurance to estimate general age trends and tailor policies accordingly. Moreover, McKinsey & Company estimated the potential of \$300-\$500 billion global savings in data warehousing expenses by utilising Hadoop in the healthcare sector.
- In finance, financial institutions consolidate data warehouses into a single repository backed by Hadoop in order to improve risk analysis models and enable customised products and services for its customers. Indeed, after the 2008 recession (6), financial institutions began transitioning to a unified Hadoop cluster and aggregating petabytes of financial data for advanced analytic techniques such as sentiment analysis for improved risk assessment. Moreover, major institutions like Morgan Stanley and JPMorgan Chase rely on Hadoop for critical investment decisions due to its scalability and low-latency processing of structured/unstructured data. Hadoop is also involved in anti-money laundering practises, asset valuation and portfolio analysis for banks and governmental financial organizations.
- In education, Hadoop is expected to facilitate digital schooling and student progress tracking through Learning Management Systems (LMS). Currently, Hadoop contributes to expediting data processes and enabling real-time Internet of Things (IoT) infrastructure projects which is likely to enhance the capabilities of educational institutions in handling and analysing vast volumes of data, leading to more personalised learning experiences and improved decision making.
- In retail, large-scale retailers utilise Hadoop in order to consolidate vast amounts of Point of Sale transactional data from various sources in order to predict demand, boost profits and create targeted marketing campaigns. Moreover, In the retail sector, Hadoop also find applications in inventory forecasting, dynamic pricing, supply chain efficiency, targeted marketing and fraud prevention. Moreover, amid increasing customer interactions across social media and retail channels, companies such as Etsy and Sear utilise retail big data analytics through Hadoop to understand customer purchase behaviour and address sales funnel leaks.
- In security, Hadoop improves national/local security by optimising the resolution of crimes across different regions and linking apparently unrelated events, which enables law enforcement authorities to quickly issue alerts to other agencies and the public. In fact, in 2013, the National Security Agency (NSA) adopted Hadoop (7) as a cost effective alternative for advanced detection capabilities in areas such as terrorism, cybercrime and other threats.

- In telecommunications, Hadoop has been utilised by several telecom companies to address challenges involving big data. In fact, China Mobile Limited utilised Hadoop to access billions of mobile call records in real-time and utilised Intel technology and HBase to store and process big data, adding nearly 30 Terabytes each month. Conversely, Nokia utilised HDFS (8) to manage 100 TB of structured data and over 500 TB of semi-structured data stemming from their manufactured mobile phones. The telecommunications industry includes various forms of communication such as local and long distance calls, wireless communications, text messaging, high-speed internet, television streaming and other satellite based communications. As a matter of fact, these activities generate vast volumes of data essential for creating marketing strategies through customer insights such as location and preferences.
- In the cloud, Hadoop has integrated into popular cloud platforms such as AWS, GCP and Azure. Indeed, this transition has introduced features such as dynamic cluster provisioning, elastic scalability and seamless integration with other cloud native tools. Moreover, cloud providers like AWS, Google Cloud and Azure offer managed Hadoop service that simplify the deployment and maintenance of clusters, allowing users to focus on data analysis. These services also ensure compatibility with other components of the Hadoop ecosystem and support integration with analytics, machine learning and data visualization tools. Recent advancements in the cloud, such as AWS Glue and Azure Synapse Analytics, have introduced serverless options for running Hadoop jobs, thus eliminating the need for explicit cluster management.

In the future, Hadoop is expected to reach \$84140 million in the Big data analytics market by 2028, with a projected CAGR of 23.9%. This growth is expected to create 11.5 million job opportunities globally by 2026 and attract interest from major companies such as OCBC Bank, Cisco, IBM, Google, Dell, Siemens and Twitter. Currently, Hadoop accounts for 19.18% of the market share in Big data analytics market and plays a crucial role in the technology market which lead to a growing global demand for Hadoop professionals, resulting in significant salary growth.

3.5 Hadoop Competitors

Apache Hadoop made a significant impact on the data analytics and data management industries. However, a number of competitors have emerged in response to various challenges faced by Hadoop such as complexity, security issues, slow performance and concurrency limitations by offering alternative approaches to data processing, data storage and data analytics. The most notable competitor being Apache Spark.

3.5.1 Apache Spark

Apache Spark is an open-source distributed processing system designed for handling large-scale data workloads. Unlike Hadoop's reliance on disk storage, Spark utilises in-memory caching and optimised query execution to enable fast analytic queries on datasets of various sizes. With development APIs supported in Java, Scala, Python and R, Spark facilitates code reuse across a range of workloads such as batch processing, interactive queries, real-time analytics, machine learning and graph processing.

While Hadoop's MapReduce serves as a robust programming model for parallel and distributed big data processing, it encounters challenges due to its sequential multi-step approach which involves repeated disk reads and writes throughout job execution.

Spark was developed to overcome these limitations by executing processing tasks in-memory, thus streamlining the efficiency of job steps and enabling data reuse across parallel operations. Indeed, with Spark, operations are consolidated into a single step where data is read into memory, operations are performed and results are written back, which results in quicker execution times.

Spark further enhances performance by leveraging in-memory caching, which facilitates fast execution of machine learning algorithms and interactive analytics tasks through DataFrames (an abstraction over Resilient Distributed Dataset (RDD) objects cached in memory). This approach substantially reduces latency, making Spark multiple times faster than MapReduce, particularly in machine learning and interactive analytics scenarios.

3.5.2 History

Apache Spark was first developed in 2009 as a research project at UC Berkeley's AMPLab, a collaborative effort involving students, researchers and faculty with a focus on data intensive application domains. The objective of Spark was to develop a framework for fast iterative processing such as machine learning and interactive data analysis, whilst also maintaining the scalability and fault tolerance of Hadoop MapReduce. The initial paper on Spark, titled "Spark: Cluster Computing with Working Sets" was published in June 2010 and the software was later released as open source under a BSD license. In June 2013, Spark entered incubation status at the Apache Software Foundation (ASF) and was officially recognized as an Apache Top-Level Project in February 2014.

3.5.3 Differences

Numerous organizations have found that Spark and Hadoop MapReduce, despite their different designs, work well together when utilised to tackle larger business challenges. Indeed, Hadoop includes the Hadoop Distributed File System (HDFS) for storage, MapReduce as its execution engine and YARN for resource management. In contrast, Spark is an open source framework designed for interactive queries, machine learning and real time workloads. Unlike Hadoop, Spark does not possess its own storage system but it conducts data analytics on various storage systems such as HDFS, Amazon Redshift or Couchbase. Moreover, Spark on Hadoop uses YARN to share the same cluster as other Hadoop engines, which ensures consistent service levels and response times.

- **Speed:** Spark utilises in-memory caching and optimized query execution techniques to perform analytic queries on data of any size, which results in significantly faster processing times compared to Hadoop. Moreover, Spark has demonstrated remarkable speed even when dealing with data stored on disk as it holds the world record for large-scale on-disk sorting.
- **Developer Friendly:** Apache Spark provides developers with a wide range of programming languages such as Java, Scala, R and Python. Additionally, Spark provides APIs designed for handling large datasets by offering a collection of over 100 operators for data transformation and familiar data frame APIs for working with semi-structured data. These APIs simplify the process for developers by abstracting the complexity of distributed processing through high-level operators, which significantly reduces the amount of code needed.
- **Multiple Workloads:** Apache Spark has the capability to handle various workloads such as interactive queries, real-time analytics, machine learning and graph processing. Indeed, it allows for the integration of multiple workloads within a single application.

3.6 Advantages/Disadvantages of Hadoop

Hadoop is a framework for distributed storage and processing of big data across clusters of computers. However, its implementation comes with both advantages and disadvantages that impact data management and data analytics

3.6.1 Advantages

- **Varied Data sources:** Hadoop excels in its versatility in handling a wide array of data types such as text files, XML documents, images, CSV files, etc., which are stored in HDFS without prior pre-processing. By handling structured (e.g. MySQL data), semi-structured (e.g. XML, JSON) and unstructured (e.g. images and videos) data efficiently, Hadoop offers flexibility in data processing, which is invaluable for enterprises analysing vast datasets from sources such as social media and email in order to extract valuable insights.
- **Cost effective:** Hadoop presents an economical solution by utilising clusters of commodity hardware for data storage which keeps the cost of adding nodes to the framework relatively low. With advancements such as the reduction of storage overhead to just 50% in Hadoop 3.0 compared to 200% in previous versions, fewer machines are needed to store data due to significantly decreased redundancy. This flexibility empowers IT specialists to select and procure various hardware types tailored to specific project needs, unlike traditional relational databases, which require costly hardware and high-end processors for big data handling.
- **Performance:** Hadoop, with its distributed processing and storage architecture, manages to process vast amounts of data at high speeds and outperforms supercomputers of 2008. It achieves this feat by effectively leveraging parallel processing on data through the map and reduce process. The HDFS achieves this by handling numerous file blocks simultaneously and surpasses traditional Database Management Systems in terms of speed and efficiency, particularly when managing large quantities of unstructured data.
- **Fault-tolerant:** Hadoop's fault tolerance strategy involved replicating data across multiple DataNodes in the cluster, with each file block being replicated three times to prevent data loss from technical issues. However, this method results in increased storage space and network bandwidth usage, effectively utilising double the resources. With Hadoop 3.0, erasure coding was introduced which improves data storage efficiency by creating additional parity blocks from existing data blocks. In this new approach, HDFS stores a combination of original data blocks and parity blocks, reducing disk usage by half compared to triple replication whilst still ensuring data resilience. In case of node failure, these parity blocks help in reconstructing lost data segments, enabling recovery without excessive redundancy.
- **Low Network Traffic:** In Hadoop, when a user submits a job, it is segmented into multiple independent sub-tasks which are then assigned to individual data nodes. This approach minimises the amount of code transferred to data nodes, thus reducing network traffic within the cluster as each data node processes only a small portion of data
- **High Throughput:** Throughput refers to the number of tasks completed within a specified timeframe and it is necessary in assessing the performance of Hadoop clusters. Indeed, by utilising its distributed file system, Hadoop segments tasks into smaller units and distributes them among data nodes within a cluster in order to achieve high throughput.
- **Scalability:** Hadoop operates on the concept of horizontal scalability, in which entire machines are added to scale the cluster instead of making changes to individual machine configurations. Its shared-nothing architecture ensures that all nodes operate independently, which enables Hadoop clusters to scale efficiently (supporting more than 10000 data nodes) without causing disruptions to the overall system. In comparison, traditional RDBMS systems frequently encounter problems in scaling effectively and managing large volumes of data.

- **Multiple Programming Languages:** While the Hadoop framework mainly utilises Java for coding, developers can also code using many other programming languages such as C, C++, Perl, Python, Ruby and Groovy. Additionally, command-line utilities are commonly scripted using shell scripts.
- **Ecosystem:** Hadoop offers a variety of tools that provide flexibility and efficiency for handling a range of tasks. At the core of Hadoop is HBase, which is a NoSQL database capable of managing large tables. Additionally, tools like Apache Flume and Apache Sqoop simplify data ingestion by aggregating log data and facilitating data transfer between Hadoop and relational databases. For exploration and analysis, tools such as Apache Pig, Apache Hive, Apache Drill, etc. use SQL-like languages that seamlessly integrate with other Hadoop components. For coordination and scheduling, Hadoop clusters are managed by tools like Apache ZooKeeper and Apache Oozie which ensure efficient management of operations and metadata. For real-time streaming analytics, tools such as Apache Kafka, Apache Spark, Apache Storm, etc. allow for the creation of real-time data pipelines. This array of tools represents only a fraction of the toolkit available for data engineering needs offered by Hadoop.
- **Compatibility:** Alongside its native HDFS, Hadoop supports a wide range of storage services and file systems such as FTP, Amazon S3 and Azure Storage. After the release of Hadoop 3, Microsoft Azure Data Lake and Aliyun Object Storage System were later integrated as well. Additionally, Hadoop is compatible with a range of processing engines which enable the integration of HDFS with Apache Spark or other computing technologies.
- **Community:** Hadoop may not have as large of a community as some top programming languages but it has a significant support base within the Big Data platform industry. Indeed, it is prominently featured on the 2022 StackOverflow list of popular frameworks and libraries, alongside other tools such as Apache Kafka and Apache Spark that are part of its ecosystem. This widespread popularity provides ample resources for Hadoop-related queries and networking opportunities for like-minded individuals on platforms such as StackOverflow, Reddit's Hadoop community and Cloudera's developer hub.
- **Open Source:** Hadoop's open-source nature allows for easy access to its source code which allows customisation in order to meet specific needs. This flexibility encourages collaboration and innovation within the Hadoop community, which leads to improvements and bug fixes in the code.
- **Hadoop Cloud Services:** Today, businesses can utilise Big Data analytics on Hadoop without the need for substantial hardware investment through Hadoop as a service (HaaS) providers. Indeed, those considering HaaS options over on-premises deployment have a variety of platforms to choose from such as Amazon EMR, Azure HDInsight, Dataproc and Oracle Big Data Service, which offer access to Hadoop in the cloud with pricing based on a per cluster/per hour model. Moreover, Amazon EMR handles large-scale data processing jobs and machine learning applications using open-source Big Data frameworks like Hadoop, Spark and Hive. Meanwhile, Dataproc, a Hadoop/Spark platform by Google, integrates with other Google Cloud services to enable MapReduce/Spark jobs to run on data stored in Google Cloud Storage. These HaaS providers not only offer Hadoop software and infrastructure but also offer cluster management, data migration services, customizable dashboards and additional security features to streamline and enhance the Big Data analytics experience for businesses.

3.6.2 Disadvantages

- **Managing Small Files:** Initially designed for large datasets, Hadoop faces challenges handling numerous small files due to its architecture being optimised for a smaller number of large files. Even with manual adjustments, managing vast numbers of small files strains the system's capabilities, as each block in the HDFS requires 150 KB of metadata storage on the NameNode. Indeed, in MapReduce, slave nodes process inputs sequentially (whether they are chunks of 128

MB or smaller) leading to notable processing delays in analysing large quantities of small files. To address this issue, one approach involves consolidating small files into a single sequence file or using the HAR (Hadoop Archives) command to bundle them into larger chunks.

- **Java Vulnerability:** Hadoop, being written in Java is susceptible to security breaches due to its inherent vulnerabilities. Indeed, Java's widespread adoption makes it an attractive target for cybercriminals which in turn increases the risk of exploitation within the Hadoop framework. This close relationship between Hadoop's architecture and Java's popularity highlights the need for robust security measures to safeguard against potential threats and breaches.
- **Processing Overhead:** In Hadoop, the processing of data involves extensive read and write operations, particularly when handling terabytes or petabytes of data. These operations are essential for managing large-scale datasets which rely on disk storage rather than in-memory calculations, which results in significant processing overhead when dealing with vast volumes of data.
- **Batch processing:** At its core, MapReduce operates solely in batch processing mode, which results in longer processing times. Moreover, its inability to cache results leads to slower performance compared to other frameworks like Spark as it requires pre-stored data files to achieve low latency output. While Hadoop 3 shows improvement, it still lags in caching and in-memory computation, making it less suitable for real-time analytics demands where timely insights are critical. To address this issue, utilising Apache Kafka for live data stream ingestion and utilising frameworks such as Apache Spark, Apache Storm or Apache Flink for real/near real-time processing proves beneficial. Notably, Apache Spark, outperforms MapReduce by a considerable margin by providing a more effective solution for time-sensitive analytics tasks whilst also accommodating batch workloads.
- **Iterative Processing:** The sequential data flow structure of Hadoop poses limitations for iterative processing or machine learning tasks as it lacks the capability to independently handle iterative processing. In contrast to the cyclic data flow typically required for machine learning, Hadoop operates with data moving through a sequence of stages, where the output of one stage feeds into the next. This limitation restricts its versatility for analytical requirements that rely on iterative processing which highlights a gap in Hadoop's capabilities for such tasks.
- **Security:** Despite data security being a top priority in Hadoop, the lack of encryption at storage and network levels raises significant security concerns. As data plays a vital role in organizational operations, it is important for users to be cautious and implement measures to address these security vulnerabilities. Moreover, while Hadoop utilises Kerberos for security, its complexity highlights the need for careful management. Additionally, the absence of storage and network encryption in Kerberos further raises concerns about data security in Hadoop setups.
- **Learning Curve:** In data analysis, SQL is the main language used, but Hadoop does not have native support for it. As a result, developers and professionals are required to program in Java to interact directly with the Hadoop Distributed File System (HDFS), create MapReduce jobs and make the most of its capabilities. However, tools such as Apache Pig, Apache Hive and Apache Drill help streamline the complexities of Hadoop by allowing the use of SQL and SQL-like languages for data manipulation.

4 Contents and Knowledge

4.1 Software Engineering

The software engineering aspect of my project influenced the final design and clarity of my code. Indeed, I utilised various software engineering practices to achieve this outcome. For this project, I opted to keep everything simple and only utilised essential tools such as git, MapReduce, noMachine for remote server access and Notion for maintaining an updated daybook and tracking progress.

I consistently committed small changes separately when utilising git as this approach allowed me to easily review my code and isolate specific changes, which enabled me to focus on one aspect of the code at a time. For instance, if I made modifications to both the diary and `RegexSearch.java`, I would commit the changes to the diary first and then commit the changes made to `regexsearch.java`.

I also made an effort to adhere to software principles such as "Don't Repeat Yourself" (DRY) and "Keep It Simple, Stupid" (KISS). Specifically, I focused on maintaining concise and clear logic within the mapper and reducer classes, such that they can be reused when necessary. In fact, I successfully reused the logic for identifying and processing content within a specific HTML tag from `WordCount.java` to `RegexSearch.java` in order to disregard any unnecessary content.

I prioritised simplicity by avoiding complex design patterns. For instance, in `RegexSearch.java`, I utilised `startPattern` and `endPattern` (each containing a div tag) to maintain clear and straightforward code. When addressing code smells, I removed any duplicate/dead code and utilised clear and meaningful variable names to improve the overall readability of the code.

By adhering to the Hadoop paradigm, I inherently followed the principles of Abstraction and Decomposition. This involved breaking down my code into mappers and reducers, with the mapper handling calculations related to mapping the data and the reducer aggregating key-value pairs emitted by the mapper. This approach improved the manageability of my code and streamlined the debugging process.

I applied principles like encapsulation within my classes to improve the efficiency of my code. By following the Hadoop paradigm, I consistently utilised the `map` and `reduce` methods to process key-value pairs. Indeed, these methods possessed built-in iterators which streamlined the data processing and minimised the need for redundant code.

I applied Inheritance on my code by extending the abstract Hadoop mapper and reducer classes and modifying them to address the specific task I was working on. This approach enabled me to utilise the existing functionality of Hadoop mappers for extracting data from input files and the automatic aggregation of key value pairs by the reducer. As a result, I did not need to write additional code for these tasks.

I implemented exception handling several times in my code. For example, in the `NuclearDecay.java` file, I utilised `NumberFormatException` to address any potential errors that may occur when parsing mass excess uncertainty values as doubles. Indeed, two non-numerical values were present in the `massExcessUncertainty` column and by utilising error handling, I successfully prevented any disruptions to my data processing.

I also utilised comments and JavaDoc documentation to improve the clarity of my code and provide explanations for complex lines of code. Indeed, comments were used to improve code readability by explaining logic within calculations concisely, whereas JavaDoc was used to provide documentation for other people to understand how to utilise classes and methods within my code without needing to examine the code too much.

When it came to software development methodology, I adopted an agile methodology in which I would develop my code, conduct testing and subsequently make improvements based on feedback received from my supervisor.

4.2 Testing

In regards to testing, I chose not to implement Test Driven Development (TDD) for MapReduce jobs due to the complexities of its application in a distributed/parallel processing environment. Indeed, the traditional TDD approach of writing tests before code is inefficient for MapReduce, as generating output requires considerable time and effort. Instead, I utilised shell scripts to streamline the execution of Hadoop jobs and compare their outputs for validation (as suggested by my supervisor). In addition, I used Excel to validate the accuracy of the statistics for problem 3 against the results I obtained through Excel formulas.

4.2.1 Shell Script

Instead, I used shell scripts for testing to simplify the process of running tests by encapsulating various Linux command lines within them and validating the final output. The shell script executed Hadoop commands to run the Java file while they also removed existing folders from the HDFS/local system to avoid any conflicts. Following each test case execution, I compared the output of the shell script with the output generated by the MapReduce job. This verification step ensured that the program functioned correctly under specific conditions and I could identify any deviations from the expected behaviour of the program. Overall, shell scripts streamlined the testing process by executing all necessary commands for running Hadoop files, which tend to be repetitive and confirming the final outputs consistency when compared to running each Hadoop command separately.

4.2.2 Excel

In the case of problem 3, I utilised basic Excel formulas to validate the accuracy of the statistics I obtained. Specifically, I used the formula `=MODE.SNGL(K:K)` to calculate and verify the mode of the `massExcessUncertainty` column. Furthermore, I utilised the Excel function `=COUNTIF(K:K,3)` to confirm the frequency of the mass excess uncertainty value 3, which matched the corresponding data from Hadoop at 1066 occurrences. In regards to the average, I utilised the formula `=AVERAGE(K:K)` to determine the mean of the `massExcessUncertainty` column. The result I obtained was 9.53632, which was consistent with the value I derived from Hadoop (albeit with some rounding). For the maximum and minimum values, I utilised the formulas `=MAX(K:K)` and `=MIN(K:K)` which resulted in respectively 25 and 3, as I expected. To calculate the median, I used `=MEDIAN(K:K)` which returned a value of 7, which was consistent with the outcome from Hadoop. In the end, using Excel formulas facilitated the comparison of outputs between Hadoop and Excel, which allowed me to validate and detect any inaccuracies in the statistics generated for problem 3.

4.3 Nuclear Decay

Radioactive decay refers to the processes in which an unstable atomic nucleus emits subatomic particles without external stimulation. This decay results in the formation of a daughter nucleus.

A nuclide refers to a specific type of atom which is characterized by the number of protons and neutrons in the nucleus, which ultimately determines the mass of the nuclide. The mass number represents the total number of protons and neutrons in the nucleus and the atomic number represents the number of protons in its nucleus.

The mass excess of a nuclide (9) is the difference between its actual mass and its mass number in atomic mass units and it serves as a predominant method for tabulating nuclear mass. It closely approximates the mass of an atomic nucleus which is primarily comprised of constituent protons and neutrons. Moreover, the mass excess is an indicator of the nuclear binding energy in comparison to the binding energy per nucleon of Carbon-12, which is the standard atomic mass unit. A negative mass excess suggests that the nucleus possesses a greater binding energy than ^{12}C , whereas a positive mass excess indicates the opposite.

Mass excess uncertainty refers to the uncertainty associated with the measurement of mass excess (the disparity in mass between a nucleus and the sum of the masses of its protons and neutrons) and it is crucial for understanding the reliability and accuracy of nuclear mass data. This uncertainty stems from the constraints of experimental methods and the complexities of nuclear interactions. Various factors can contribute to this uncertainty such as measurement inaccuracies, limitations in equipment and uncertainties in theoretical models used in nuclear structure calculations.

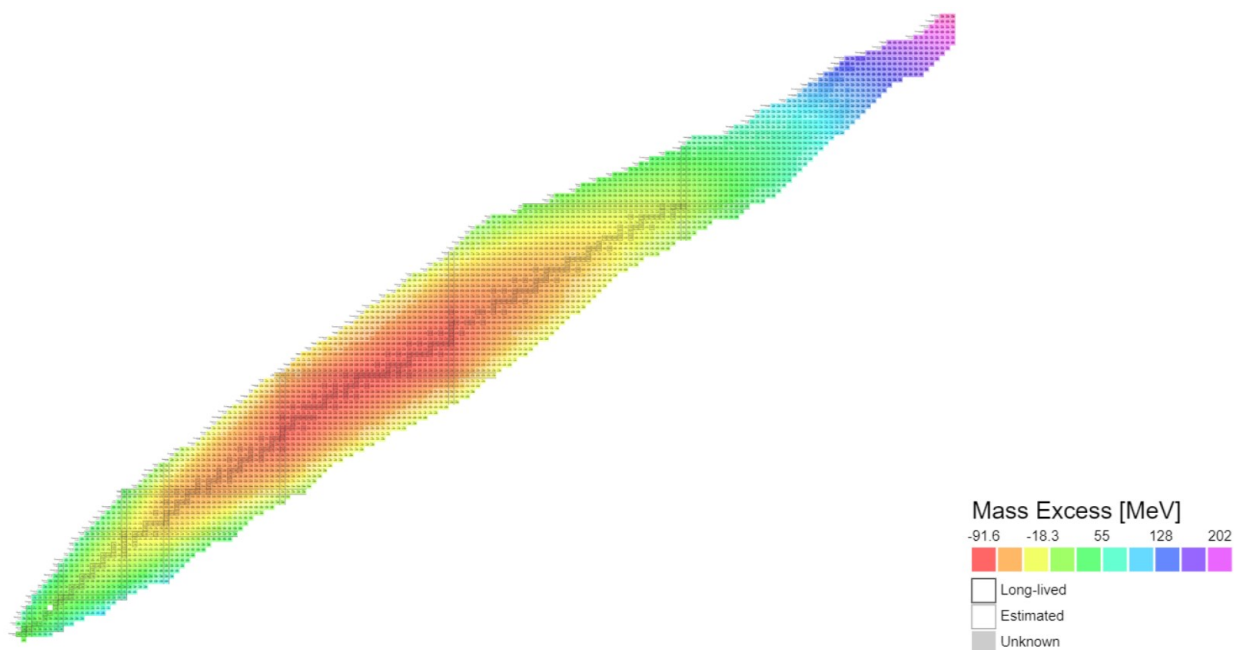


Diagram of the Mass Excess in a Nuclide Chart

4.4 Financial Concepts

4.4.1 RSI

The Relative Strength Index (RSI) (10) is a momentum indicator commonly used in technical analysis to evaluate the speed and magnitude of recent price changes in a security/stock. This helps in determining potential overvalued or undervalued conditions in the security's price.

In addition to recognizing overbought and oversold securities, the RSI can also signal potential trend reversals or corrective price movements, which helps in decision-making for buying and selling a particular security. Moreover, a reading of 70 or higher on the RSI indicates an overbought situation while a reading of 30 or lower suggests an oversold condition.

Additionally, the average gain or loss in this calculation represents the average percentage change over a specified period of time. It is worth noting that a positive value is used for the average loss. Indeed, when calculating the average gain, periods of price decrease are treated as zero whereas periods of price increase are treated as zero as well when calculating the average loss. The RSI is calculated utilising the following formula:

$$RSI = \frac{100}{1 + \frac{n_{up}}{n_{down}}}$$

where n_{up} represents the average of n-day closes and n_{down} represents the average of n-day down closes

4.4.2 Price Change

A price change (11) in the financial markets refers to a fluctuation in the value of a security/asset (either increase or decrease) from a stock's closing price on a trading day to its closing price on the previous trading day (the open). Though price changes can be computed for various time frames, the most commonly analysed is the daily price change in finance, which denotes the difference in a security's price from the previous trading day's close to the present day's close (this is the calculation I conducted for problem 4). The Price Change is calculated utilising the following formula:

$$PC = C - O$$

where C represents the closing price and O represents the opening price

4.4.3 Trading Range

The trading range (12) refers to the difference between the low and high prices for a security or index over a specific time frame such as a trading day. In technical analysis, the trading range is observed when a security maintains consistent high and low prices over a period of time. Indeed, the upper end of a security's trading range typically acts as price resistance whereas the lower end usually offers price support. When a stock breaks through or falls below its trading range, it often indicates a building momentum which is either positive or negative. A breakout occurs when the price of a security exceeds its trading range while a breakdown happens when the price falls below the trading range. The Trading Range is calculated utilising the following formula:

$$TR = H - L$$

where H represents the high price and L represents the low price

4.4.4 Volume/VROC

Volume is an important indicator that reflects the total number of shares traded within a specific time frame such as a trading day regardless of whether they were bought or sold. Typically, high volume signifies active trading in the market while low volume indicates less activity. Indeed, analysing the volume is a popular method for identifying trends and patterns in the stock market.

The Volume Rate of Change (VROC) (13) is an indicator that assesses the rate of change in trading volume over a specified period, which represents the percentage change in current volume compared to volume from a certain number of periods ago. By analysing the VROC, market participants can gain insights into the strength of a price trend. Indeed, this indicator allows them to evaluate the momentum behind price movements and ascertain whether a security is experiencing a rising or declining volume trend. A rising VROC indicates increasing buying pressure, which suggests a potential rising price trend. On the other hand, a downward trend in VROC indicates increasing selling pressure, which implies a potential declining price trend.

The Volume rate of change is calculated utilising the following formula:

$$VROC = \frac{V - pV}{pV} * 100$$

where V represents the current volume and pV represents the previous volume from n periods ago

4.5 Map and Reduce

MapReduce is a programming paradigm within the Hadoop framework designed for processing large volumes of data stored in the Hadoop Distributed File System (HDFS). It enables parallel processing by dividing large volumes of data into small chunks which are then processed simultaneously on Hadoop servers. The results are then combined to provide a unified output for the application. An essential aspect of MapReduce involves two fundamental functions which are executed sequentially:

- The Map function retrieves key-value pairs from the disk, it processes them and it then generates a new set of intermediary key-value pairs.
- The Reduce function also processes key-value pairs as input and it also outputs key-value pairs.

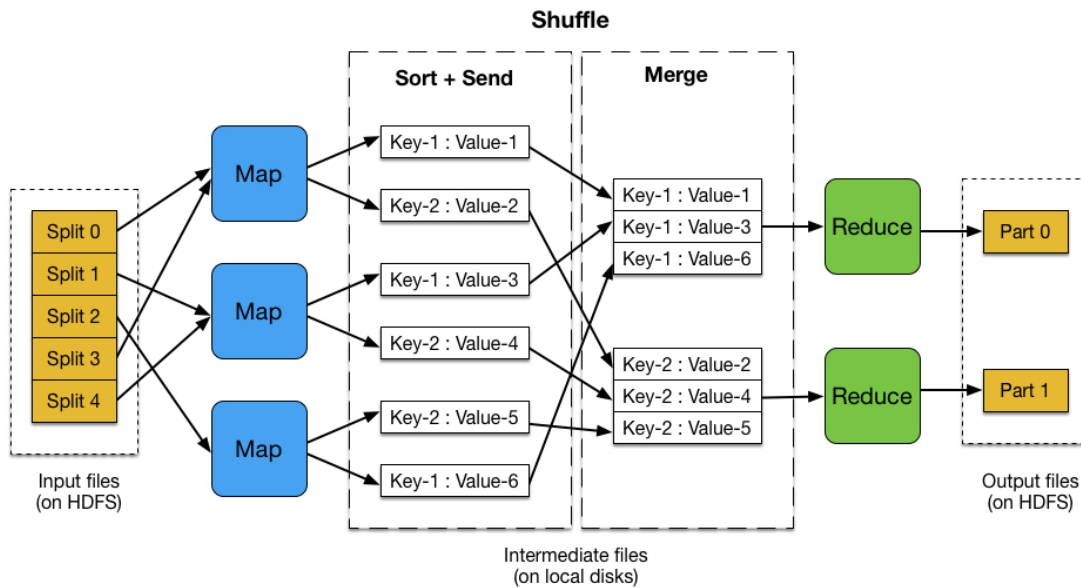


Diagram of the Functionality of Map and Reduce

4.5.1 Map & Reduce

The input data is segmented into smaller blocks and it is distributed to individual mappers for processing. Once all the mappers have finished processing, the framework will shuffle and sort the key-value pairs before sending them to the reducers. It is important to note that a reducer cannot initiate until a mapper has finished its task. Once the key-value pairs reach the reducer, it will then aggregate the values for specific keys and generate an output.

4.5.2 Combine & Partition

In the data processing pipeline between the Map and Reduce phases (14), there are two intermediate steps.

- Combining is an optional step in the data processing process. The combiner helps to streamline the data on each mapper before sending it to the reducer by making the tasks of shuffling and sorting more efficient as it reduces the data volume. In some cases, the combiner class is designated to be the same as the reducer class. However, if necessary, the combiner can also be defined as a separate class.
- Partitioning is the process of converting the key-value pairs created by mappers into a new set of key-value pairs for input to the reducer. The default partitioner calculates a hash value for the key generated by the mapper and then assigns a partition based on this hash value where each reducer has its own partition. Once the partitioning process is complete, the data from each partition is directed to a reducer.

4.6 Garbage Collection Time

Garbage collection (15) in Java is the automated process of removing unnecessary/unused code. Typically, Java applications are compiled into bytecode that is executed by a JVM, generating objects on the heap (a memory space for dynamic allocation), which are then monitored and managed by garbage collection operations. The garbage collector utilises a mark-and-sweep algorithm to identify unreachable objects and mark them for collection, it then scans live objects to determine which are still reachable, in order to ensure efficient memory management and prevent memory leaks. During this process, the collector systematically scans different sections of the heap, removing objects without references to free up memory until all unused objects are successfully reclaimed.

4.7 Parallelism

Parallel processing is a computing technique in which multiple data processing tasks occur simultaneously through numerous central processing units (CPUs) working concurrently, which can significantly reduce the execution time of a program.

At a fundamental level, the distinction between parallel and serial operations is based in the manner in which registers are utilised. In serial operations, shift registers process each bit individually, whereas registers with parallel loading can process all bits of a word simultaneously.

Typically, when processing tasks in parallel, a large project is divided into smaller tasks that are more compatible with the number, size and type of available processing units (often microprocessors). Once the tasks are allocated, each processor independently works on their assigned portion without direct communication in which they rely on software to coordinate and monitor the progress of their tasks. After completion of all program components, the outcome is a fully processed program segment.

In general, when processing tasks in parallel, a large project is segmented into smaller tasks that are compatible with the quantity, dimensions and qualities of the available processing units (typically microprocessors). After the tasks are allocated, each processor works independently on their designated segment without direct communication, but instead, they rely on software to coordinate and supervise the task progress. After the completion of all program components, the outcome is the aggregation of the individually processed segments into the final result for the project.

5 Project Progression

For my project, I decided to utilise the big data server provided by my university rather than using my personal laptop. First and foremost, the university servers offer significantly greater computational power and memory capacity compared to my laptop, which would allow me to efficiently process and analyse large datasets. Indeed, given the specifications of my laptop, I would likely struggle to handle the data processing required for tasks like data analysis. Using the command "lshw" to analyse the hardware specification revealed to me that the university server is equipped with 125 GB of RAM as well as multiple Intel Xeon E5-2683 v4 CPUs clocked at 2.10GHz. These high-performance server-grade processors are specifically designed to handle intensive workloads effectively and its multiple cores are capable of parallel processing. Additionally, the 10Gbit/s Ethernet interface (eth0) indicates that the server is capable of high-speed data transfer. Moreover, university servers have professional IT professionals for support who may help me with any problems regarding the usage of the servers and in the case of failure, they may know what happened and this would be understood in the case of my final year project.

5.1 Problem 1

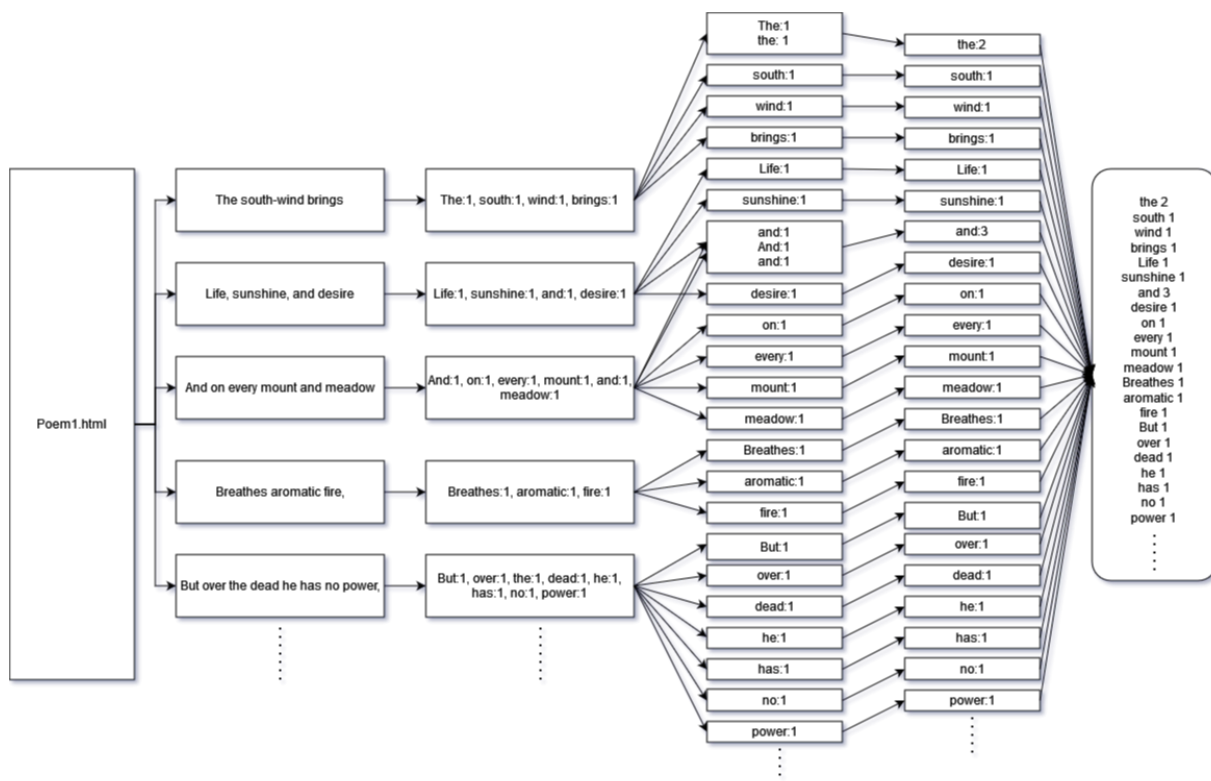


Diagram of WordCount.java processing poem1.html

In addressing Problem 1, which involved extracting and counting word occurrences from a collection of HTML pages using Hadoop MapReduce, I made a foolish mistake during my second week where I used Hashmaps in my code instead of the appropriate Hadoop methods, which arose due to communication problems with my supervisor and my limited experience with Hadoop Mapper/Reducer methods. Fortunately, I later received valuable feedback from my supervisor, who provided detailed instructions on how to utilise the mapper and reducer methods in Hadoop MapReduce and how to execute the java file using some specific commands. This feedback was helpful as it redirected me from using basic Hashmaps to adopting the more appropriate MapReduce paradigm and due to this reason, I successfully managed to develop a Java program that utilised mapper and reducer methods to generate an output displaying the number of occurrences for each word within a simple text file. This process has significantly improved my understanding of the Mapper/Reducer methods and it gave me confidence with programming in MapReduce, which I was quite unfamiliar with before. As a side note, my git repository was still not available by the first week which delayed some of my progress.

The following week, I made notable progress as I adapted the program to accept an HTML page (poetry-related) as input and output the word occurrences within the poem, which is encapsulated within 2 div tags. By this point, I have finished the majority of Problem 1, albeit I made minor refinements later in order to optimise the efficiency of my code.

```
1  import java.io.IOException;
2
3  import javax.naming.Context;
4
5  import org.apache.hadoop.conf.Configuration;
6  import org.apache.hadoop.fs.Path;
7  import org.apache.hadoop.io.IntWritable;
8  import org.apache.hadoop.io.LongWritable;
9  import org.apache.hadoop.io.Text;
10 import org.apache.hadoop.mapreduce.Job;
11 import org.apache.hadoop.mapreduce.Mapper;
12 import org.apache.hadoop.mapreduce.Reducer;
13 import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
14 import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
```

I initiated the process by creating a class named WordCount where I imported a few essential Hadoop libraries for the configuration/execution of a Hadoop MapReduce job. Noteworthy among these libraries is org.apache.hadoop.conf.Configuration, which imports the "Configuration" class from Hadoop and it configures Hadoop jobs by providing a set of properties and settings. Then, I imported Job class from the MapReduce framework which configures job-related parameters, including input/output formats and the mapper/reducer classes. Moreover, one of the main import statements are the ones designated for the Mapper and Reducer classes, which import the abstract Mapper/Reducer class from Hadoop MapReduce and serve as the foundation for the mapping and reducing phases for the Hadoop job. Furthermore, I imported FileInputFormat and FileOutputFormat which are required for the successful execution of WordCount.java since they are base classes for file-based input/output formats that are used to set the input and output paths for the MapReduce job.

```
16 public class WordCount {
17     public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable> {
```

Within the WordCount class, I implemented the TokenizerMapper and IntSumReducer classes, both inheriting from the mapper and reducer classes. In the Mapper class, I implemented TokenizerMapper to take an 'Object' and a 'Text' as key-value pairs and emit a 'Text' and an 'IntWritable'. I opted to use 'Object' as a generic key type as it would be easier to use, while selecting 'Text' over String as it is more efficient and flexible for representing textual data in Hadoop. For outputs, 'Text' was used to hold individual words and IntWritable was used to represent the count of occurrences for each word.

```
public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable> {
    private final static IntWritable one = new IntWritable(1);
    private Text wordText = new Text();
    private boolean div = false;
```

Later, I initialised a final static IntWritable variable named "one" with an initial value of 1 representing the count of occurrences for each word in the input data. By designating it as static and final, I ensured it is consistent throughout the program's execution and shared across all instances of the TokenizerMapper class. Additionally, I declared a variable of type 'Text' named "wordText" to store the individual words extracted from the input data. At the same time, I initialised a boolean variable named "div" to false. This variable serves to track whether the current line being processed resides within the two div tags encapsulating the poem ("

" and "</div>") and filter out unnecessary content.

```
public void map(Object key, Text value, Context context) throws IOException, InterruptedException {
    String line = value.toString();
    if (line.matches(".*<div class=\"post_description\">.*")) {
        div = true;
    }
    if (line.trim().equals("</div>")) {
        div = false;
    }
    if (div) {
        String[] words = line.split("\\s+");
        for (String word : words) {
            word = word.replaceAll("<[>]*>", "");
            word = word.replaceAll("[a-zA-Z]+", "");
            word = word.replaceAll("^href\\S*", "");
            String wordLowerCase = word.toLowerCase();
            wordText.set(wordLowerCase);
            context.write(wordText, one);
        }
    }
}
```

I declared a map method with three parameters: Object key, Text value and Context context. In this context, 'Object' key serves as the input key, 'Text' value represents a line of text from the input data and 'Context' context enables interaction between the mapper and the Hadoop framework for the emission of key-value pairs. Additionally, I declared that the map function may potentially throw IOException and InterruptedException, as certain operations within the map function, such as writing to the context or the handling input/output parameters, have the potential to throw exceptions.

```
String line = value.toString();
if (line.matches(".*<div class=\"post_description\">.*")) {
    div = true;
}
if (line.trim().equals("</div>")) {
    div = false;
}
if (div) {
    String[] words = line.split("\\s+");
    for (String word : words) {
        word = word.replaceAll("<[>]*>", "");
        word = word.replaceAll("[a-zA-Z]+", "");
        word = word.replaceAll("^href\\S*", "");
        String wordLowerCase = word.toLowerCase();
        wordText.set(wordLowerCase);
        context.write(wordText, one);
    }
}
```

This section of the code is responsible for the main processing of input data, focusing on identification, cleaning and extraction. It involves evaluating each line to determine if it falls between two div tags ("

" and "</div>") and utilising regex expressions, the content is cleansed and the words are subsequently extracted. The resulting key-value pairs denote the occurrence of each word. The process begins with the initialisation of a variable named "line" of type String, designed to store the 'Text' value (representing a line of text from the input data) in a String format. This conversion is necessary to perform string manipulation operations on the data later on.

```
if (line.matches(".*<div class=\"post_description\">.*")) {
    div = true;
}
if (line.trim().equals("</div>")) {
    div = false;
}
```

I then implemented two conditional statements to assess if the current line contains a specified HTML tag. Indeed, the variable "div" changes its state accordingly. If the line matches ".<div class=\"post_description\"", the "div" variable becomes 'true'. Conversely, if after trimming the whitespaces, the line contains "</div>", then "div" changes to 'false'.

```
if (div) {
    String[] words = line.split("\\s+");
    for (String word : words) {
        word = word.replaceAll("<[^>]*>", "");
        word = word.replaceAll("[^a-zA-Z]+", "");
        word = word.replaceAll("^href\\S*", "");
        String wordLowerCase = word.toLowerCase();
        wordText.set(wordLowerCase);
        context.write(wordText, one);
    }
}
```

This section of the code does most of the data processing for Problem 1. Indeed, it executes exclusively when the variable "div" is set to 'True', which would indicate that it encountered the div tag "<div class=\"post_description\">". Then, each line is splitted into an array of words based on whitespace. For each word, a series of regular expressions are applied to clean it up using the replaceAll() method, this involves removing HTML tags (<[^>]*>), non-alphabetical characters ([^a-zA-Z]+) and words starting with "href" (^href\\S*) (I used RegExr to derive with these expressions). The cleaned word is then converted to lowercase to ensure case-insensitive counting, stored in the variable "wordLowerCase" and set as the output key "wordText." Ultimately, the output, consisting of "wordText" and the count "one," is emitted for further processing by the reducer method.

```
public static class IntSumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    private IntWritable count = new IntWritable();
```

For the Reducer phase in the MapReduce process, I defined a nested static class named "IntSumReducer" which extends the Reducer class provided by Hadoop, accepting a 'Text' as the input key and an 'IntWritable' as the input value (these key-value pairs originate from the Mapper class). In this Reducer class, I instantiated an IntWritable variable named "count" with a new "IntWritable()" object which accumulates the sum of values associated with a specific key during the reduce phase. In the end, the output key-value pairs maintain the same type as the input pairs, as the Reducer aggregates identical key-value pairs emitted by the Mapper to produce the final output.

```

public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException, InterruptedException {
    int sum = 0;
    for (IntWritable val : values) {
        sum += val.get();
    }
    count.set(sum);
    context.write(key, count);
}

```

In the `IntSumReader` class, I introduced a void method named "reduce" which aggregates values associated with a specific key. This method takes three parameters: a 'Text' key representing the aggregation key, an 'Iterable' collection of `IntWritable` values corresponding to word occurrences and a 'Context' object for interacting with the Hadoop framework. Moreover, the reduce method declares an `IOException` which addresses potential issues that may arise during I/O operations, specifically when utilising the `context.write` method for file system interactions. Furthermore, `InterruptedException` is also declared in case a thread is interrupted while in a waiting state, typically timeouts or errors within the Hadoop ecosystem.

```

int sum = 0;
for (IntWritable val : values) {
    sum += val.get();
}

```

I then initialised an integer variable named "sum" to 0, which serves for tracking the count of word occurrences. Also, I utilised a for loop to iterate through the values in the "iterable," which contains all instances of the word in question. Within the loop, I invoked the `get()` method to retrieve the integer value stored in each `IntWritable` so that they can later be aggregated in sum.

```

count.set(sum);
context.write(key, count);

```

In the end, I utilised `count.set(sum)` to set the cumulative count as the 'value' for the current key (count representing an instance of `IntWritable`). Subsequently, I emitted the key-value pair, maintaining the original key (representing the unique word) while updating the 'value' to reflect the total occurrences of that word.

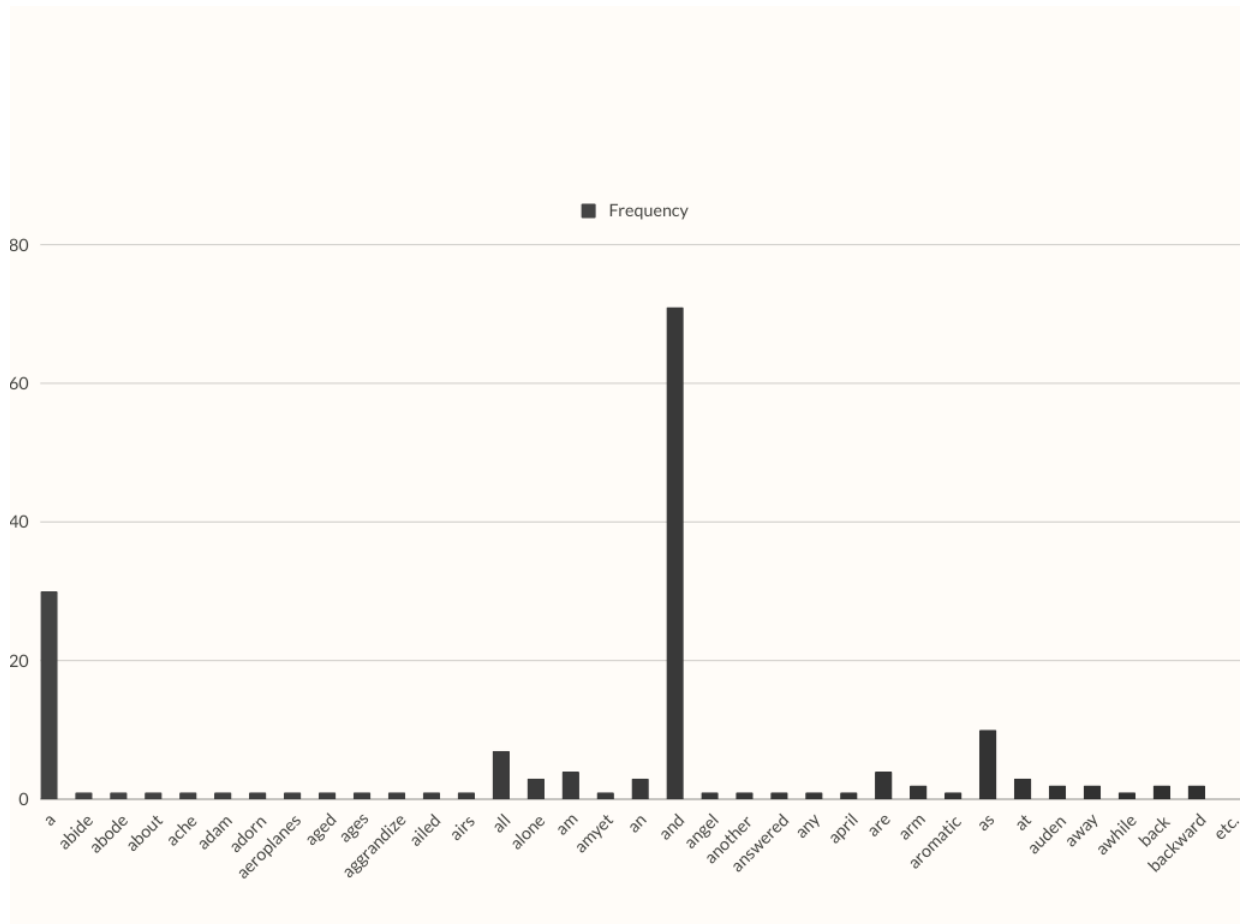
```

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "wordcount");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenzierMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.setInputPaths(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}

```

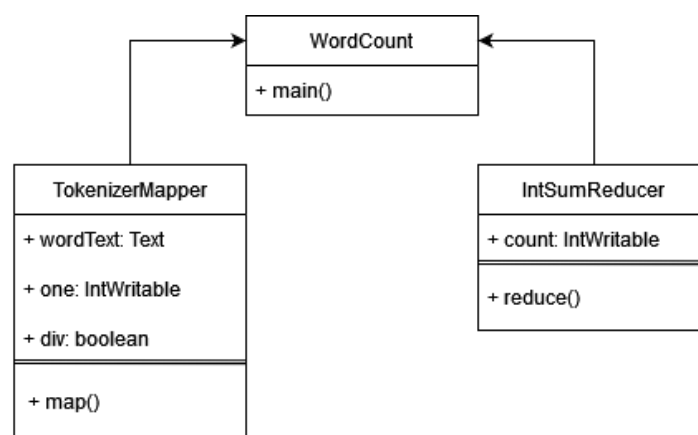
In the main method, I configured the job configuration by creating a `Configuration` object (conf) to hold the Hadoop job settings. Then, I set up a Hadoop MapReduce job named "wordcount" using this configuration which contains important parameters such as file system details, input/output formats and compression settings. Following this, I specified the main classes containing the Mapper and Reducer class. Then, I configured the mapper, combiner and reducer for the job. In the end, I defined the output key and value types, as well as the input and output paths. In order to execute the

job, I used `job.waitForCompletion(true)`, which outputs progress information to the terminal during execution, indeed, it returns true if the completion is successful and false otherwise. The `System.exit()` line terminates the job and it assigns a status code of 0 for success and 1 for failure.



Visualisation of the output generated by the processing of the "poems" folder

5.1.1 Class Diagram WordCount.java



5.2 Problem 2

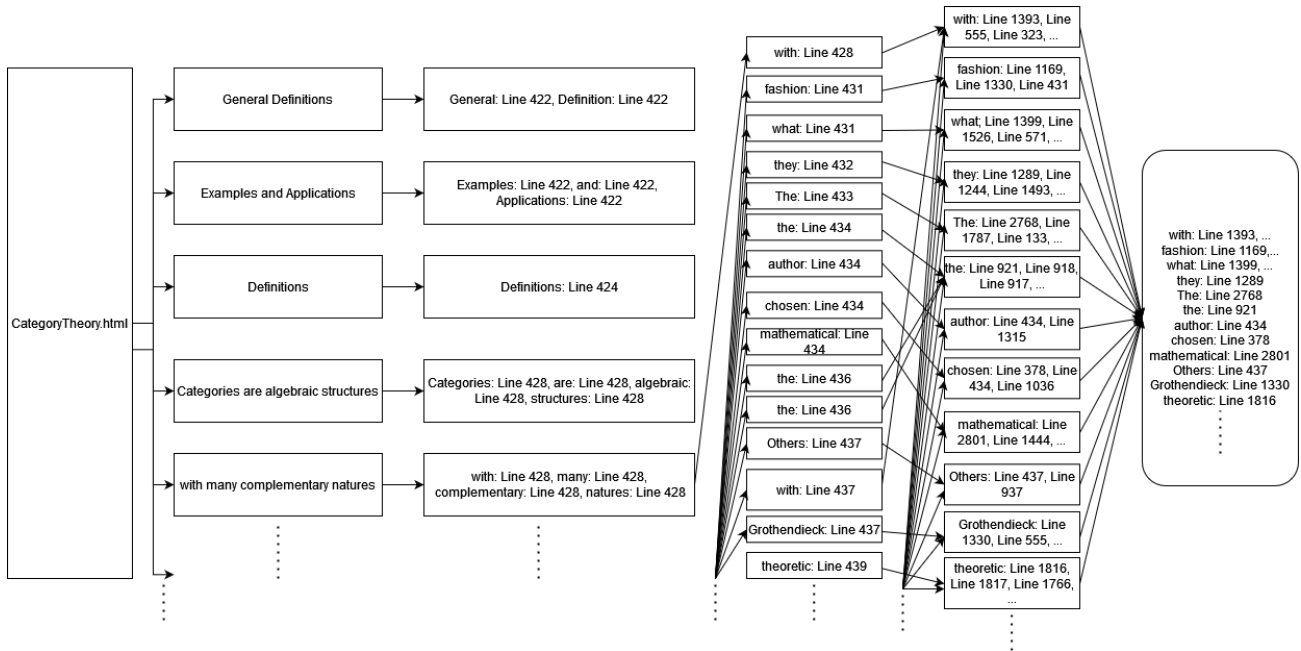


Diagram of `RegexSearch.java` processing `CategoryTheory.html`

In addressing Problem 2, the objective was to create a MapReduce program for extracting specific content matching a regex expression from philosophy-related HTML files and outputting a list of tuples each of which consisted of the matching string and its location in the input text. I chose a regex expression that aimed to identify words containing the character "h", with the program outputting each matching word alongside its line number. Initially, I began working on the mapper and reducer classes trying to create something basic and indeed, I eventually managed to create a mapper to find the occurrences of the specified pattern. However, I made a critical mistake in the calculation of line numbers. In fact, instead of utilising the Byteoffset key provided by `TextInputFormat`, I incorrectly implemented a variable called "linenumber" and a while loop which would iterate through all the matches and then increment the linenumber in the end, resulting in the incorrect calculation of the number of occurrences for matches rather than the line numbers. Over the subsequent weeks, I refined the code to process a folder of HTML pages related to philosophy and the updated implementation successfully outputted words containing "h" based on the regex expression along with their line numbers.

```
public class RegexSearch {
    public static class SearchMapper extends Mapper<LongWritable, Text, Text, Text> {
```

I started off by creating a Java file named "RegexSearch" and within this file, I defined a nested "SearchMapper" class which extended the native Hadoop abstract Mapper/Reducer classes. The class processes key-value pairs of type `LongWritable` (the Byte offset) and `Text` (the line of text). Then, it emits the key-value pairs of `Text` (the matched Strings) and `Text` (location of the said Strings in the text file).

```
private Text locationText = new Text();
private boolean divTag=false;
private Pattern startPattern = Pattern.compile("<div id=\"main-text\">");
private Pattern endPattern = Pattern.compile("<div id=\"bibliography\">");
```

I then defined four private variables to identify specific patterns within the text file. Initially, I declared a private instance named "locationText" of type "Text", intended to represent the line number. Following this, I initialised a boolean variable named "divTag" to false in order to track the processing state of content within the main text section (enclosed by two <div> tags). Lastly, I initialised two private objects of type Pattern named "startPattern" and "endPattern," each configured with regular expressions corresponding to the opening tag <div id="main-text"> and the closing tag <div id="bibliography">, which are used to process data within the specified tags through logic involving the boolean variable "divTag."

```
@Override
public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
    Matcher startMatcher = startPattern.matcher(value.toString());
    Matcher endMatcher = endPattern.matcher(value.toString());
    Configuration conf = context.getConfiguration();
    String regexPattern = conf.get("regex pattern");
```

Subsequently, I created a map method designed to process each line of the input data and is equipped with three parameters: a LongWritable key, representing the byte offset of the input data; a 'Text' value, representing a line of text from the input data and a "Context" context object, providing access to the Hadoop configuration. The map method is designed to handle potential IOException and InterruptedException scenarios related to data reading/writing errors or thread interruptions. Within the map method, I declared two Matcher objects named "startMatcher" and "endMatcher". These objects use "startPattern" and "endPattern" to identify the initiation and conclusion of a specific section within the input text. Ultimately, I obtained the configuration settings from the Hadoop MapReduce job's configuration by retrieving the value associated with the key "regex pattern" through the use of conf.get(). This was done in order to compare it with the content of the input text data.

```
if (startMatcher.matches()) {
    divTag = true;
}
if (endMatcher.matches()) {
    divTag = false;
}
if (divTag) {
    Matcher contentMatcher = Pattern.compile(regexPattern).matcher(value.toString());
    while (contentMatcher.find()) {
        String matchedString = contentMatcher.group();
        locationText.set("Line " + key);
        context.write(new Text(matchedString), locationText);
    }
}
```

In the next phase, I developed the main section of the code which is responsible for managing the logic to identify the start and end of a specific content section within the input text. The process involves examining each line of text and determining matches on certain patterns, specifically startPattern and endPattern. Upon a match with startPattern, which marks the initiation of the desired text section, the boolean variable divTag is set to true. Conversely, if a match with endPattern occurs, it indicates the conclusion of the desired content (main body of text) and divTag is set to false. When divTag is true (indicating the current line is within the desired content section), the mapper proceeds to extract all the relevant information. Moreover, a new Matcher object named contentMatcher is initialised

with "regexPattern" which is designed to match specific patterns within the text content extracted from the input data. Subsequently, I initiated a while loop to search for words matching the regular expression from contentMatcher, which upon finding a match, a String variable is created containing the match using the .group() method. Then, the line number key is concatenated with the word "Line" in locationText. In the end, the key-value pair, consisting of the matched string (the word matching the regular expression) and locationText (line number), is emitted for further processing.

```
public static class SearchReducer extends Reducer<Text, Text, Text, Text> {
    public void reduce(Text key, Iterable<Text> values, Context context) throws IOException, InterruptedException {
```

In the Reducer class, I extended the Reducer abstract class provided by Hadoop in order to create the SearchReducer class, which processes input key-value pairs, where the key is a Text representing the word matched by the regular expression and the value is a Text representing the line number. Moreover, the SearchReducer class includes a reduce method that takes three parameters: A 'Text' key representing the word identified by the mapper, A Iterable<Text> values representing the collection of line numbers associated with the identified word and 'Context' context which provides access to ArrayList<String> uniqueValues = new ArrayList<>(); method. Also, within this method, I implemented the logic for (Text value : values) { associated with each word.

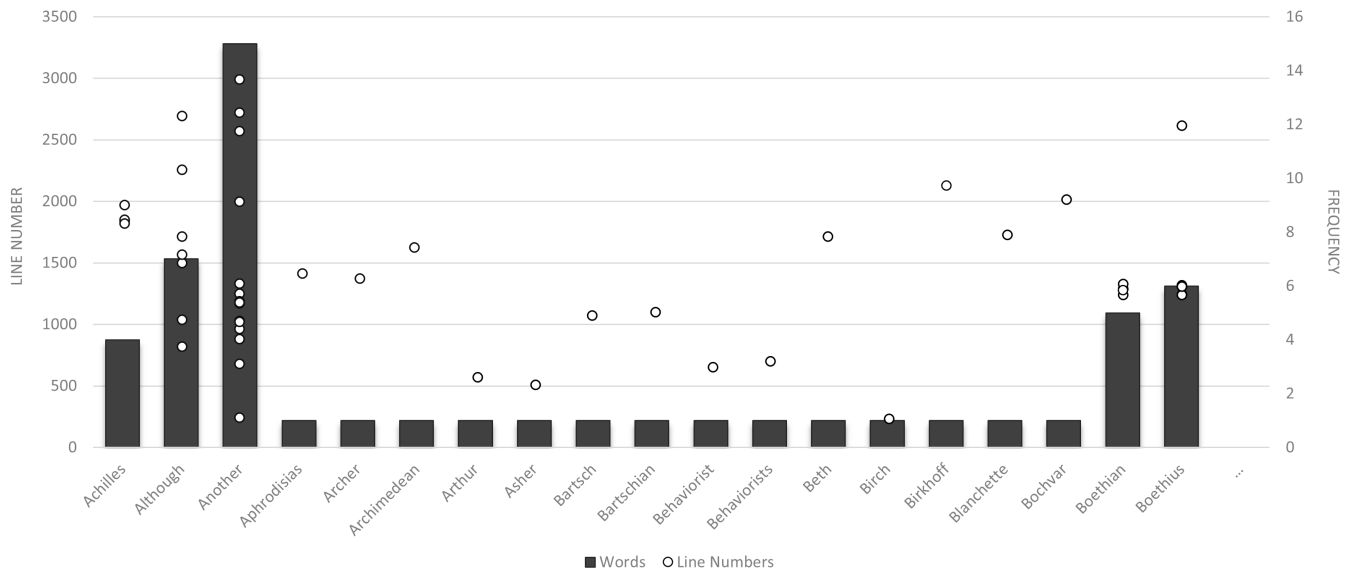
```
    String valueString = value.toString();
    if (!uniqueValues.contains(valueString)) {
        uniqueValues.add(valueString);
        locations.append(valueString).append(", ");
    }
}
if (locations.length() > 0) {
    locations.deleteCharAt(locations.length() - 2);
}
context.write(key, new Text(locations.toString()));
```

I then instantiated a StringBuilder object named "locations" to concatenate line numbers where the matched string was present in the text. Additionally, I declared an ArrayList of type 'String' named "uniqueValues" to store unique values encountered during the iteration process. During this iteration, which loops through values associated with a specific key, I created a temporary string named valueString to store the current value. If valueString is not present in uniqueValues, it is then added to the list and the StringBuilder is updated by appending the value followed by a comma for formatting. Moreover, to ensure clarity, if there already existed values in the StringBuilder for one specific key, the last two characters (comma and space) are removed. In the end, the key (representing the matched string) and its corresponding location are outputted using context.write.

```
public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    conf.set("regex pattern", args[2]);
    Job job = Job.getInstance(conf, "Regex Search");
    job.setJarByClass(RegexSearch.class);
    job.setInputFormatClass(LineNumberInputFormat.class);
    job.setMapperClass(SearchMapper.class);
    job.setReducerClass(SearchReducer.class);
    job.setMapOutputKeyClass(Text.class);
    job.setMapOutputValueClass(Text.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);
    FileInputFormat.setInputPaths(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
```

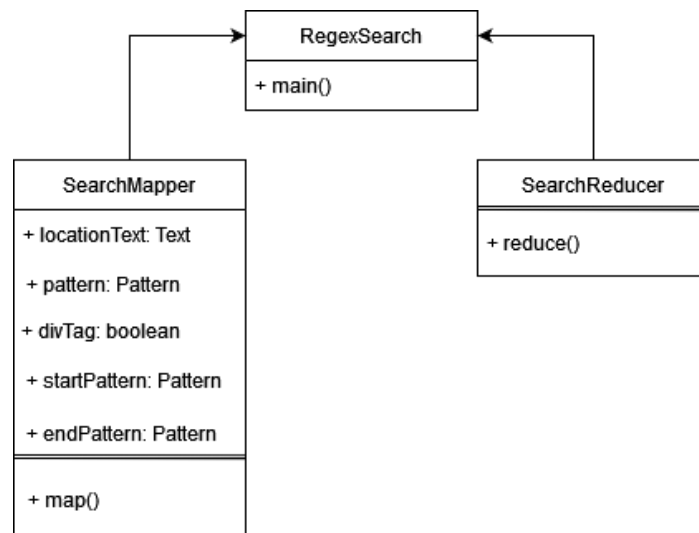
In the main method, I initiated the configuration of the MapReduce job by declaring a Configuration object named "conf" to store important parameters. Afterwards, I saved a regular expression pattern in args[2] (an additional argument in the command line) that will be utilised for searching strings within the input text data. Following this, I set the input format class for the job to LineNumberInputFormat which is a custom input format that overrides the default TextInputFormat and provides more

functionality by incorporating a custom record reader (LineNumberRecordReader). This record reader assigns line numbers to each input record instead of the usual ByteOffset. Furthermore, I specified the Mapper and Reducer classes for the job and set the input and output paths for it.



Visualisation of the frequency and line locations of the words generated by processing the philosophy folder

5.2.1 Class Diagram RegexSearch.java



5.3 FileInputNumberFormat

Due to the limitations of TextInputFormat, which provides the ByteOffset output for the current line, I opted to modify the original abstract class to derive the line number by borrowing and modifying code (16) I found online, which assigns numbers to each record. Through the use of online resources (17) (18), including Stack Exchange forums, I modified the TextInputFormat file to output the line number. Given the difficulty and uncertainty in working on a class inherited from TextInputFormat, I heavily relied on external sources and borrowed/modified code (19) (20) as it was not initially a focal aspect of the main four problems for my project.

```
public class LineNumberInputFormat extends FileInputFormat<LongWritable, Text> {
```

Initially, I wrote a class named "LineNumberInputFormat" which extends the original TextInputFormat class and accepts a LongWritable object (indicating the line numbers) and a Text object (representing the lines) as inputs.

```
@Override
public RecordReader<LongWritable, Text> createRecordReader(InputSplit split, TaskAttemptContext context) throws IOException, InterruptedException {
    return new LineNumberRecordReader();
}
```

Then, the "createRecordReader" method was overridden to provide a custom record reader tailored for the input format. Specifically, the overridden method was modified to return an instance of LineNumberRecordReader, which is an inner class defined within LineNumberInputFormat. The "createRecordReader" method takes as an input an InputSplit named "split" (representing the input data to be processed by the mapper) and a TaskAttemptContext called "context" (providing the context for task attempts) as parameters.

```
public static class LineNumberRecordReader extends RecordReader<LongWritable, Text> {
    private LineRecordReader lineRecordReader;
    private long lineNumber;
```

Then, the class named "LineNumberRecordReader" which implements the "RecordReader" interface was instantiated, as it takes a 'LongWritable' and a 'Text' as input parameters. Within this class, two private variables were declared: "LineRecordReader" which is an instance of the standard LineRecordReader, responsible for reading lines from the input split and "lineNumber" which is a long variable utilised to store the current line number.

```
@Override
public void initialize(InputSplit split, TaskAttemptContext context) throws IOException, InterruptedException {
    lineRecordReader = new LineRecordReader();
    lineRecordReader.initialize(split, context);
    lineNumber = 0;
}
```

In the LineNumberRecordReader class, the "initialize" method was overridden, which is a part of the RecordReader interface. The purpose of this override is to initialise the record reader before processing any key-value pairs. Then, an instance of the standard Hadoop LineRecordReader was created and initialised with the input split and context. Additionally, the lineNumber was set to 0 in order to track the line numbers assigned to each record.

```

@Override
public boolean nextKeyValue() throws IOException, InterruptedException {
    boolean hasNext = lineRecordReader.nextKeyValue();
    if (hasNext==true) {
        lineNumber++;
    }
    return hasNext;
}

```

Then the nextKeyValue method, which is responsible for advancing the next key-value pair in the input split, was implemented. Indeed, I introduced the boolean variable hasNext which manages the transition to the next key-value pair through the lineRecordReader.

```

@Override
public LongWritable getCurrentKey() throws IOException, InterruptedException {
    return new LongWritable(lineNumber);
}

@Override
public Text getCurrentValue() throws IOException, InterruptedException {
    return lineRecordReader.getCurrentValue();
}

@Override
public float getProgress() throws IOException, InterruptedException {
    return lineRecordReader.getProgress();
}

@Override
public void close() throws IOException {
    lineRecordReader.close();
}

```

In the end, the last four methods were overridden. The initial method, getCurrentKey(), yields the current key as a LongWritable. The second method, getCurrentValue(), invokes the current LineRecordReader to obtain the content of the current line as a Text object. The third method, getProgress(), derives progress information from the LineRecordReader through the input split. Lastly, the fourth method, close(), is executed when the record reader is no longer needed.

5.4 Problem 3

5.4.1 Description

In problem 3, I was tasked with conducting basic statistical analysis from a nuclear decay dataset containing mass excess uncertainty data. My objective was to calculate key statistical measures such as the mean, mode, median, minimum and maximum values of the dataset. Additionally, I generated a frequency distribution of the data at the 9.1% percentile and visualised the shape of the distribution using the Matplotlib library in python in order to gain insights into certain tendencies present in the dataset.

I began by examining the dataset and identifying columns suitable for statistical analysis (I specifically focused on columns that did not contain any missing values). As a result, I selected the massExcessUncertainty

column, which represents the variation related to the difference in mass between a parent nucleus and its decay products.

I began by calculating each statistic in separate Java files to simplify testing and debugging for each specific variable. I initially chose this approach because I was unsure of how to calculate all the variables in a single Java file. However, while most statistics were straightforward to calculate, determining the mode proved to be a bit challenging. Initially, I was unsure how to calculate the mode using only the map and reduce methods, which would have allowed its integration with other statistical calculations in a single Java file. Nonetheless, after completing each individual calculation, I consolidated them into a single Java file called "NuclearDecay.java". In this file, I used the `cleanup()` method (which I discovered after discussions with some peers) to streamline the output of the statistical data and prevent any confusion.

After completing the calculations, I proceeded to generate the frequency distribution for problem 3. I began by determining the cumulative frequencies of the values in the `massExcessUncertainty` column by using the code from a previous commit where I attempted to calculate the mode. I decided to display the frequency distribution at 9.1% percentiles based on the fact that the range was 22 and a 2 KeV interval would have been practical. However, calculating at 10% percentiles would have resulted in a 2.2 KeV interval, which would not only complicate calculations but also be impractical as mass excess uncertainty is calculated in integers.

In the end, I utilised the Python library "matplotlib" to create a histogram displaying values from the `massExcessColumn` at 2 KeV intervals. However, due to my limited familiarity with the library, I encountered some challenges with the plotting process and due to this reason I borrowed a code snippet (21) for determining the bar length/interval. After the visualization of the data on a graph, several insights can be derived from the data.

5.4.2 Technical Decisions

In my code, the Mapper class processes input data and generates key-value pairs for statistical calculations, while the Reducer class combines these values to compute statistical measures such as the mean, mode, median, etc. Moreover, I included error handling in the mapper phase to handle `NumberFormatExceptions`, which displays a message to identify problematic values if a mass excess uncertainty value is not in numerical form before being parsed as a double. Furthermore, when addressing problem 3, I initially chose to calculate each statistic in separate Java files and then merge them into a single Java file afterwards. I adopted this approach for easier debugging, testing and organising the logic for each statistic in separate files, rather than dealing with everything in a single file which can lead to difficulties when managing different key-value pairs aggregated by the reducer.

5.4.3 Dataset

I selected a Nuclear Decay dataset as the main dataset for this project based on my interest in the subject matter. Nuclear decay datasets often contain intricate metrics (many of which have values that are very small) which posed an interesting challenge for me for statistical analysis. Additionally, I wanted to integrate physics and mathematics into problem 3, so I chose a nuclear decay dataset that would allow me to conduct basic statistical analysis to gain insights in the context of physics.

5.4.4 Code

In this section, I will be addressing each section of the code for `NuclearDecay.java`. However, I will not delve into the individual Java files I used for each statistic calculation in order to avoid any convolution on my report. Moreover, the basic structure for computing these statistics within the separate Java files has been maintained in `NuclearDecay.java`.

```
public class NuclearDecay {
    public static class NuclearMapper extends Mapper<LongWritable, Text, Text, DoubleWritable> {
        private DoubleWritable occurrence = new DoubleWritable(1.0);
```

I began the problem by creating a Java file named "NuclearDecay.java". Within this file, I extended the native Hadoop abstract Mapper/Reducer classes to "NuclearMapper" and "NuclearReducer". The NuclearMapper class takes a LongWritable and a Text as input key-value respectively (where the input key represents the offset of the beginning of the line from the start of the file and the input value represents the line itself) and outputs a Text and a DoubleWritable as output key-values (where the Text represents the type of statistic being calculated and the DoubleWritable represents the value itself under the massExcessUncertainty Column). I also declared a private variable named "occurrence" of type DoubleWritable which is initialised with the value 1.0 and it is used to represent the occurrence count of each mass excess value.

```
@Override
public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
    String[] columns = value.toString().split(",");
```

I defined a map method that handles each line of input data and it utilises three parameters: a LongWritable key, which represents the byte offset of the input data; a Text value, which represents a line of text from the input data; and a "Context context" object, which allows access to the Hadoop configuration. Following this, I defined a String array named columns, which splits the input text line into an array of strings using a comma as the delimiter (each element in the array separated by commas represents a single field from the input line).

```
if (columns.length >= 11) {
    String massExcessColumn = columns[10];
```

Next, I utilised a conditional statement to verify if the current data line contains a minimum of 11 columns, which ensures that there are enough columns for the mapper to process the necessary data. Subsequently, I assigned the value of the 11th column to a String variable named "massExcessColumn" to store specific mass excess uncertainty values.

```
try {
    double massExcessValue = Double.parseDouble(massExcessColumn);
    context.write(new Text(massExcessColumn), occurrence);
    context.write(new Text("Mean: "), new DoubleWritable(massExcessValue));
    context.write(new Text("Minimum: "), new DoubleWritable(massExcessValue));
    context.write(new Text("Maximum: "), new DoubleWritable(massExcessValue));
    context.write(new Text("Median: "), new DoubleWritable(massExcessValue));
} catch (NumberFormatException e) {
    System.out.println("There was a problem with the value: " + "\\\" + massExcessColumn + "\\\"");
}
```

I then implemented try and catch statements to handle any potential NumberFormatException errors that may arise while converting a string to a double in the massExcessColumn. Indeed, I created a double variable named massExcessValue to store the numerical equivalent of the mass excess, which is obtained by parsing the "massExcessColumn" string as a double. I then emitted the massExcessColumn value along with a count of its occurrence ("1") as a key-value pair for calculating the mode. Additionally, key-value pairs were also emitted for statistics like Mean, Maximum, Minimum, etc. with massExcessValue as the value.

```
public static class NuclearReducer extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {
    private double minimumValue = 99.0;
    private double maximumValue;
    private double sum = 0.0;
    private int count = 0;
    private List<Double> valuesList = new ArrayList<>();
    private Map<String, Double> mode = new HashMap<>();
    private double maxCount;
    String maxKey = null;
}
```

In the Reducer class, I defined NuclearMapper by extending the abstract class Reducer provided by Hadoop. Initially, I initialized several variables including a private double named "minimumValue" set to 99.0 for calculating the minimum and a double named "maximumValue" (with a default of 0.0) for calculating the maximum. Additionally, a double sum and an integer count were initialised for calculating the mean, a maxCount for determining the mode and a maxKey initially set to null but later assigned the massExcessUncertainty value with the highest cumulative frequency. I also declared an ArrayList of doubles named "valuesList" for storing values for the median calculation and a Hashmap named "mode" for tracking the frequency of mass excess uncertainty values.

```
@Override
public void reduce(Text key, Iterable<DoubleWritable> values, Context context) throws IOException, InterruptedException {
    for (DoubleWritable value : values) {
        double currentValue = value.get();
        if (key.toString().equals("Minimum: ") && currentValue < minimumValue) {
            minimumValue = currentValue;
        }
        if (key.toString().equals("Maximum: ") && currentValue > maximumValue) {
            maximumValue = currentValue;
        }
        if (key.toString().equals("Mean: ")) {
            sum += currentValue;
            count++;
        }
        if (key.toString().equals("Median: ")) {
            valuesList.add(currentValue);
        }
        if (!(key.toString().equals("Median: ")) && !(key.toString().equals("Mean: ")) && !(key.toString().equals("Maximum: ") || key.toString().equals("Minimum: "))) {
            mode.put(key.toString(), mode.getOrDefault(key.toString(), 0.0) + currentValue);
        }
    }
}
```

I then implemented a reduce method that takes a Text key and an iterable of DoubleWritables. Within this method, I utilised a for loop to iterate through all the values associated with the input key where each value is converted into a double using 'value.get()' and stored in a variable named 'currentValue'. Following this, I analysed the key to determine the type of statistic it represents, in order to ensure the accurate calculation of statistics and to prevent any errors from occurring.

If the key is "Minimum:", the minimumValue variable is updated if "currentValue" is smaller than the current minimumValue. If the key is "Maximum:", the maximumValue variable is updated if "currentValue" is greater than the current maximumValue. For the key "Mean:", the "currentValue" is added to the sum variable and the count is incremented. In the case of "Median:", the "currentValue" is added to an ArrayList called "valuesList". If the key does not match any of the predefined keys ("Median:", "Mean:", "Maximum:", "Minimum:"), the mode map is updated by including the mass excess value and adding the "currentValue" to the existing value if the key is already present. However, if the key is not present, it is then initialized with the "currentValue".

```

for (Map.Entry<String, Double> entry : mode.entrySet()) {
    if (entry.getValue() > maxCount) {
        maxKey = entry.getKey();
        maxCount = entry.getValue();
    }
}
}

```

Next, i iterated through the entries of the mode Hashmap and for each entry, I checked if the value associated with the current key is higher than the maximum count (the value with the highest cumulative frequency). If the value is indeed greater, I update the maxKey to the key of the current entry and update the maxCount to be the value associated with that key.

```

@Override
protected void cleanup(Context context) throws IOException, InterruptedException {
    if (count > 0) {
        double mean = sum / count;
        context.write(new Text("Mean: "), new DoubleWritable(mean));
    }
    Collections.sort(valuesList);
    int size = valuesList.size();
    double median;
    if (size % 2 == 0) {
        median = (valuesList.get(size / 2 - 1) + valuesList.get(size / 2)) / 2.0;
    } else {
        median = valuesList.get(size / 2);
    }
    String modeOutputKey = maxKey.replace("Mode: ", " ");
    context.write(new Text("Mode: " + modeOutputKey), new DoubleWritable(maxCount));
    context.write(new Text("Median: "), new DoubleWritable(median));
    context.write(new Text("Minimum: "), new DoubleWritable(minimumValue));
    context.write(new Text("Maximum: "), new DoubleWritable(maximumValue));
}

```

Finally, I utilised a cleanup method to complete the tasks after all key-value pairs associated with a given key have been processed by the reducer, as this allows for the final computations to be performed based on the accumulated data. Initially, I checked to ensure that the count is greater than 0; if so, the mean is then calculated by dividing the sum by the count and it is emitted through the context.write() method. Following this, the array list "valuesList", which contains all the values for median calculation, is sorted using Collections.sort(). Subsequently, a double named "median" is initialized to store the median value and the size of "valuesList" is stored in an integer variable called "size". To calculate the median, the program checks if the size of the list is even; if it is, the median is calculated by averaging the two middle values. On the other hand, if the size of the list is odd, the middle value is taken directly as the median.

Next, I assign a variable named modeOutputKey to remove the prefix "Mode: " in the 'maxKey' string in order to address an issue where the "Mode: " prefix is duplicated in the output. Following this adjustment, the remaining statistics are outputted as key-value pairs, with the key representing the type of statistic and the value representing the actual data associated with that statistic.

5.4.5 Parallelism

For problem 3, parallelism can be utilised to optimise the calculation of certain order statistics such as the maximum or minimum values. This can be achieved by dividing the dataset into partitions and simultaneously identifying the maximum or minimum values within each partition. Ultimately, the highest/lowest value among all the maximum/minimum values in each partition will be determined as the final maximum/minimum for a given dataset.

5.4.6 Frequency Distribution

```
public class NuclearDecayDistribution {
    public static class DistributionMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
        private Text massExcessUncertainty = new Text();
        private IntWritable occurrence = new IntWritable(1);
```

I began by creating a public class called `NuclearDecayDistribution` in order to calculate the cumulative frequencies for each value in the `massExcessUncertainty` column. Within this class, I created the mapper/reducer classes which are `DistributionMapper` and `DistributionReducer`. The `DistributionMapper` generates a key of type `Text` for the mass excess uncertainty value and an `IntWritable` value to indicate a single occurrence of that value. Additionally, I defined two private variables in this class: `massExcessUncertainty` (of type `Text`) to store the extracted mass excess uncertainty value from the input data and `occurrence` (of type `IntWritable`) to track the occurrences of each unique mass excess uncertainty value encountered in the input data. The `occurrence` variable is initialised with a count of 1 for each instance.

```
@Override
public void map(LongWritable key, Text currentline, Context context) throws IOException, InterruptedException {
    String[] columns = currentline.toString().split(",");
    if (columns.length >= 11) {
        try {
            massExcessUncertainty.set(columns[10]);
            context.write(massExcessUncertainty, occurrence);
        } catch (NumberFormatException e) {
            System.out.println("There was a problem with the value: " + "\\\" + massExcessUncertainty + "\\\"");
        }
    }
}
```

I then created the `map` method within `DistributionMapper`. In this method, I established an array of `String` variables called `columns` to store the specific data column from the input file using the `"toString()"` method to split the current line by commas (`,`).

Next, the method verifies if there are more than 11 columns to ensure that there are at least 11 elements in the array before accessing the 11th element, which is assigned to `massExcessUncertainty`. Subsequently, the mapper emits key-value pairs where the key represents the `massExcessUncertainty` value and the value denotes the occurrence of this particular mass excess uncertainty value.

```
public static class DistributionReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    private final IntWritable mode = new IntWritable();
    @Override
    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException, InterruptedException {
        if (!key.toString().equals("AP") && !key.toString().equals("massExcessUncertainty")) {
            int sum = 0;
            for (IntWritable value : values) {
                sum += value.get();
            }
            mode.set(sum);
            context.write(key, mode);
        }
    }
}
```

The `DistributionReducer` class extends the native `Reducer` class which provided by Hadoop. It

processes the key-value pairs emitted by the mapper as input and produces output key-value pairs. The main difference is that the output value represents the total cumulative frequency for a specific mass excess uncertainty value.

In this class, I have initialized a variable named `cumulativefreq` of type `IntWritable`, which stores the total number of occurrences for a mass excess uncertainty value.

In the Reducer class, I have implemented the `reduce` method. This method checks if the key is not "AP" or "massExcessUncertainty" (the only two non-numerical values found in the `massExcessUncertainty` column) in order to exclude them from the final results. I have also declared an `int` variable named `sum`, which is initialized to 0 and used to sum the total occurrences of a particular mass excess uncertainty.

After the summation, the total count is stored in the `cumulativefreq` variable. Finally, the method emits the key-value pair where the key represents the mass excess uncertainty and the value represents its total cumulative frequency.

5.4.7 FrequencyDistribution.py

To create the frequency distribution file for problem 3, I used Python and the `matplotlib` library which was recommended by my supervisor. Given my limited experience with `matplotlib`, I borrowed the structure of a code snippet (22) from the internet to help with constructing the frequency distribution into ranges. I deviated from the suggested 10% percentile approach and decided to use 9.1% percentiles for easier interval calculations as mentioned earlier.

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

I began by importing `matplotlib`, a plotting library, which I followed by utilising the command `matplotlib.use("Agg")` in order to enable the generation of plots directly from the Linux command line, which is also beneficial for testing purposes. Next, I imported `pyplot` with the common alias `plt`, which enables me to customize my plots.

```
data = {}
with open("NuclearDecayDistributionOutput/part-r-00000", "r") as file:
    for line in file:
        line = line.strip()
        if "\t" in line:
            key, value = map(int, line.split("\t"))
            data[key] = value
```

Next, I initialised an empty dictionary called "data" and opened the file ("part-r-00000") in read mode which is the output file for the frequency distribution. I then utilised a for loop to iterate through each line of the file and applied the method `.strip()` to eliminate any whitespaces, ensuring that only alphanumeric words were retained. If a tab character was detected, I split the line into two parts and assigned them to integer variables named `key` and `value`. It is important to note that the output file contains entries in the format (`{massExcessUncertaintyvalue}; {cumulativefrequency}`). The key-value pairs were then added to the data dictionary, with the value stored alongside the corresponding key.

```
frequency_distribution = {}
for i in range(3, 26, 2):
    frequency_distribution[i] = sum(data.get(j, 0) for j in range(i, i+2))
x_bar = [(range(3, 26, 2)[i] + range(3, 26, 2)[i+1]) / 2 for i in range(len(range(3, 26, 2)) - 1)]
bar_values = [frequency_distribution[x] for x in range(3, 26, 2)[-1]]
```

I then created an empty dictionary named `frequency_distribution` to store the frequency of occurrence for each range of mass excess uncertainty. I proceeded to iterate from 3 to 35, with an interval of 2. This interval was determined to be the optimal interval based on my calculations, as it resulted in whole number increments. During each iteration, the frequency for the current range was calculated by summing up the frequencies of mass excess values within that range. A list called `x_bar` was utilised to store the midpoint of each range, calculated by finding the average of the lower and upper bounds of each range. Finally, `bar_values` was used to store the frequency values corresponding to each range, retrieved from the `frequency_distribution` dictionary.

```
plt.figure(figsize=(8, 6))
plt.bar(x_bar, bar_values, width=1.2, color= "#2a2a2a")
plt.xticks(range(3, 26, 2))
plt.xlabel("Mass Excess Uncertainty (in atomic mass units u)")
plt.ylabel("Cumulative Frequency")
plt.title("Frequency Distribution for the Mass Excess Uncertainty of Isotopes of Elements")
plt.savefig("frequency_distribution.png")
```

In the end, I personalized my plot by adjusting the figure size and creating a bar chart using the `bar()` function. The `x_bar` variable was utilised for the x-coordinates, while the `bar_values` variable was used to calculate the corresponding bar values. I then opted to set the position of the ticks on the x-axis at intervals of two and proceeded to label both the x and y axes of the graph. Lastly, I added a title to my graph and saved it as "frequency_distribution.png".

5.4.8 Interpretation

Given the nature of the dataset, it is challenging to infer information from the results presented in the frequency distribution graph. Indeed, this difficulty arises from the lack of detailed information on the methodology utilised during the experimentation for gathering the nuclear decay data. Albeit not mandatory, I included an interpretation section for problem 3 due to a conversation I had with my supervisor, who was curious about the peak displayed in the interval 19-21 KeV in the frequency distribution graph.

When analysing the frequency distribution graph, it is apparent that as the mass excess uncertainty value increases, its frequency of occurrence decreases. One possible factor for this feature could be a systematic error in the measurement in the 19-21 KeV interval, which may have impacted the accuracy of mass excess uncertainties and eventually lead to a concentration of data points within that range. Indeed, calculating the mass excess uncertainty is challenging as there exists a significant margin of error due to limitations in current technology.

The higher concentration within this interval may also be influenced by theoretical extrapolation. In the article "The AME 2020 atomic mass evaluation" (23), theory is in fact used to predict the mass excess uncertainty values alongside a margin of error. In fact, the article states:

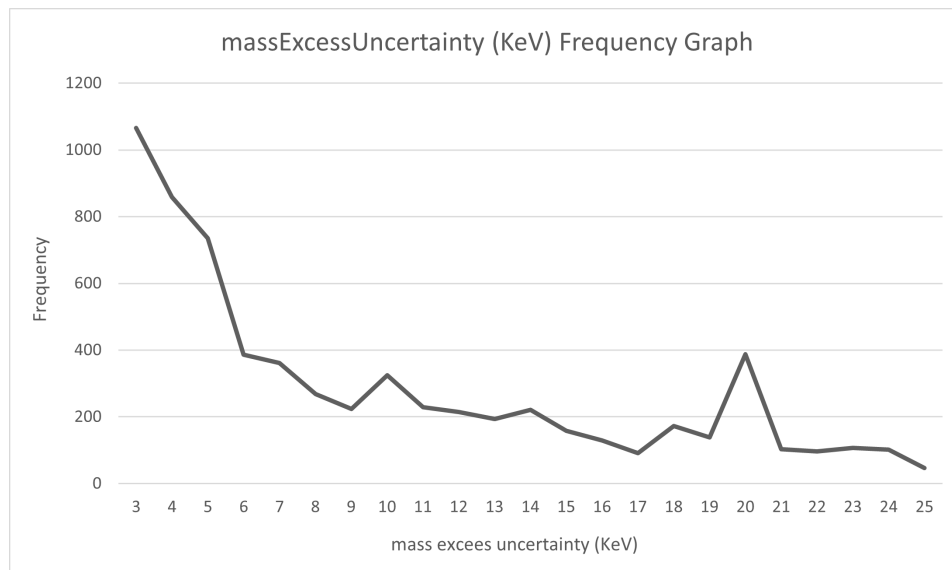
"Mass excess Mass excess, in keV and its uncertainty (one-standard deviation). In cases where the furthest-left significant digit in the uncertainty was larger than 3, values and uncertainties were rounded off, but not to more than tens of keV. (Examples: $2345.67 \pm 2.78 \rightarrow 345.7 \pm 2.8$, 2345.67 ± 4.68

$\rightarrow 2346 \pm 5$, but $2346.7 \pm 468.2 \rightarrow 2350 \pm 470$).

in place of decimal point: value and uncertainty derived not from purely experimental data, but at least partly from TMS”

This indicates that theory is utilised to predict the value of the mass excess uncertainty, rather than relying solely on experimental data. This implies that certain nuclides or nuclear decay processes may possess unique characteristics which leads to mass excess uncertainties falling within this particular range. However, its challenging to determine the reason for such an occurrence as not much research has been conducted in this area.

In the end, although I can speculate about the factors leading to the peak in the frequency distribution graph, it is impossible for me to definitively determine the cause. This is due to the lack of details regarding the experimentation procedure and the limited information about nuclide properties.



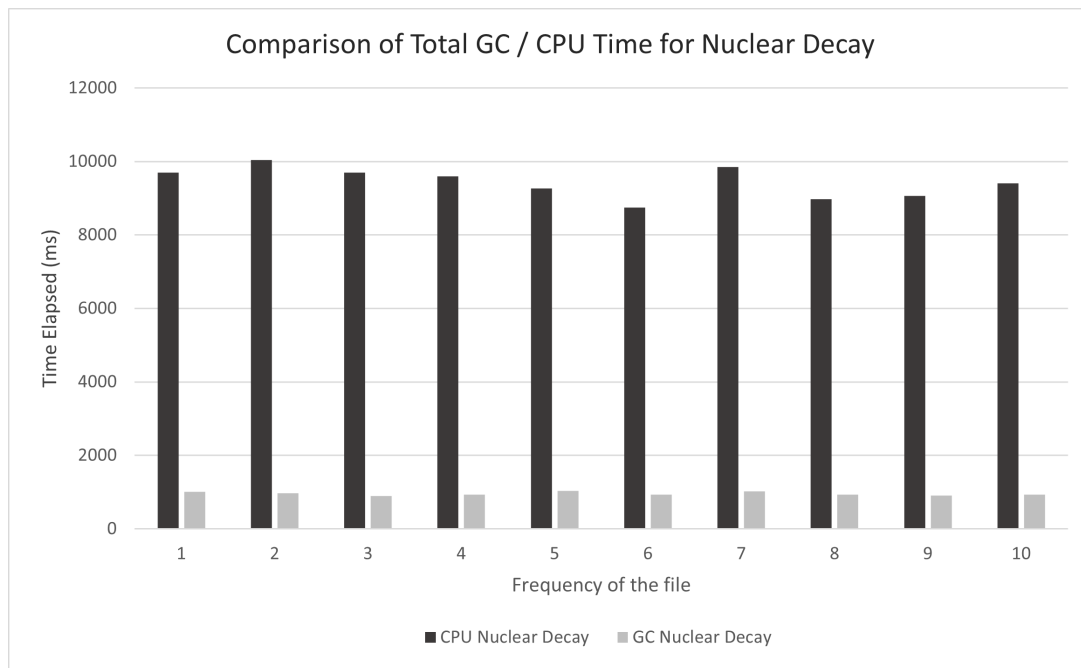
Frequency Graph for Mass Excess Uncertainty

5.4.9 Performance Analysis

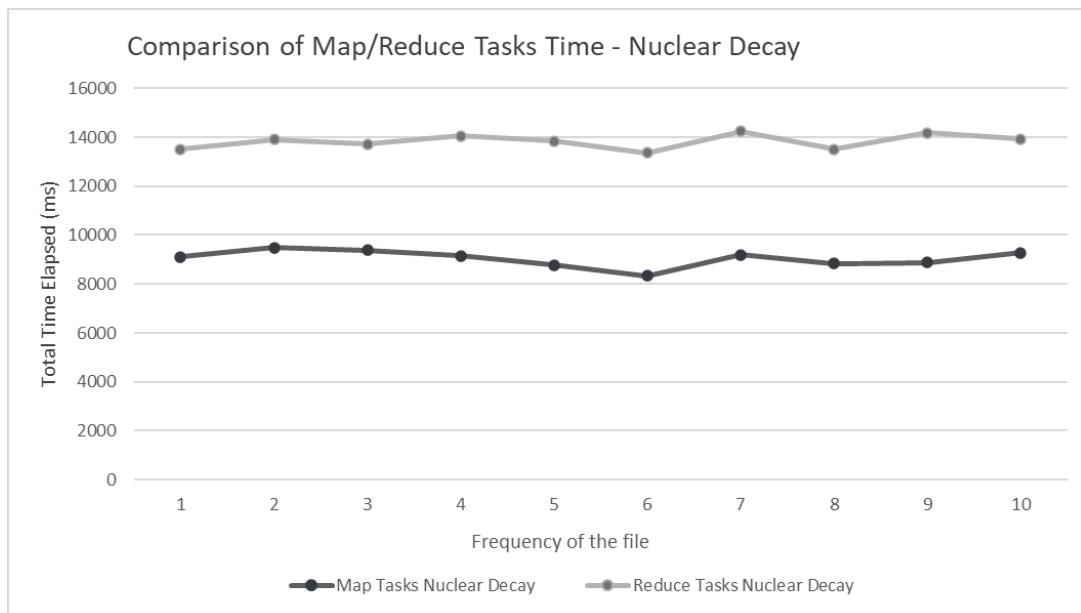
Performance analysis is an essential aspect in optimizing Hadoop algorithms to ensure they meet efficiency standards. Indeed, it offers valuable insights into areas of improvement and bottlenecks by examining the execution time, resource utilization and overall throughput of a system (reviewing the logs). This process involves the measurement, monitoring and analysis of the behaviour of Hadoop applications and the Hadoop cluster hardware across a range of workloads and conditions.

An important point to note is that for problem 3, I would only conduct performance analysis solely on NuclearDecay.java. This is because the calculations performed in NuclearDecayDistribution.java are already somewhat incorporated in NuclearDecay.java through the calculation of the mode. Moreover, NuclearDecayDistribution.java primarily includes code regarding the computation of the frequency distribution whereas NuclearDecay.java calculates the order statistics, which is the main objective of problem 3.

I began by running Hadoop jobs on a directory containing a dataset file and documenting the GC/CPU time, map/reduce tasks time and maps/reduces in occupied slots time in an Excel spreadsheet. After reviewing these statistics, I would upload the dataset file again to the same directory to run the same job on both copies of the dataset and after the completion of each job, I would record the statistics. Indeed, I would repeat this process until the Hadoop job had processed 10 copies of the same dataset file.



In terms of performance, there are noticeable variations in CPU time for the NuclearDecay.java program. Indeed, it is clear that the optimal number of csv datasets for processing is 6, as at this point the CPU time reaches a minimum time of 8740ms. Moreover, before reaching this point, the CPU time shows a gradual decrease, but it suddenly increases afterwards. This suggests that 6 datasets is the most efficient input for processing. When analysing the garbage collection (GC) time, it can be observed that there is minimal fluctuation in the time. The GC time shows a constant fluctuation and with a standard deviation of 48.9 and a mean of 952.9, it can be seen that the GC time is relatively stable, regardless of the volume of datasets being processing.



As opposed to map/reduce task times in other Java files, the map/reduce task time for NuclearDecay.java displays an interesting feature, that is, the reduce task time always exceeds the map task time. This suggests that the workload is more balanced distribution between the mapper and reducer, where in fact, the reducer plays a more significant role in which it performs more calculations. Indeed, this could be attributed to the use of hashmaps for the mode calculation in NuclearDecay.java, which

could be contributing to the longer reduce times. Additionally, the complex calculations required for deriving the order statistics are a key factor in the longer reduce task time. Moreover, when analysing the graph, it becomes apparent that there is a noticeable decrease in time at 6 datasets, with a sudden increase at 7 datasets, which indicates that the Hadoop cluster's parallel processing capability is optimised for handling 6 datasets in this scenario.

5.5 Problem 4

5.5.1 Description

I adapted problem 4 in order to conduct analysis of financial data on a NASDAQ dataset for a particular stock in order by deriving several key metrics.

Firstly, I calculated the Price Change by computing the difference between the opening and closing prices of stocks for each trading day. This calculation elucidates the directional movement and magnitude of price shifts within the specified time frame.

Secondly, the Trading Range indicator is derived by assessing the disparity between the highest and lowest prices at which a stock is traded during a single trading session. This metric serves as a measure of volatility, with wider trading ranges indicative of heightened market uncertainty and narrower ranges suggesting stability or consolidation.

Thirdly, the Volume Rate of Change is ascertained by computing the percentage change in trading volume over a predetermined period (daily basis in my case). This indicator provides insights into shifts in market participation and investor sentiment, with rapid changes in trading volume accompanying price movements often signaling the strength or weakness of prevailing trends.

Lastly, the Relative Strength Index (RSI) is calculated to gauge the speed and magnitude of price fluctuations. RSI values, ranging from 0 to 100, help identify potential overbought or oversold conditions in a stock's price, offering valuable insights into potential market reversals or continuation patterns.

Upon deriving these metrics, I analysed the results to infer information about market trends and investor behavior.

5.5.2 Dataset

I selected a NASDAQ dataset for problem 4 based on my interest in finance and mathematics. With a background in Mathematics Computer Science, I believed I could apply mathematical skills to this project in an interdisciplinary manner. However, time constraints and the nature of the dataset limited my ability to utilise all my skills, as finance approaches from a mathematical perspective often involve stochastic processes reliant on functions. Nonetheless, my foundational knowledge proved valuable for interpreting information derived from calculated metrics. Additionally, as I am interested in pursuing a master's degree in computational finance or financial mathematics, I aimed to gain a better understanding of relevant theories by focusing on a finance-related topic for problem 4.

5.5.3 Parallelism

In problem 4, parallelism can be utilised in various scenarios. One such scenario involves calculating price changes or trading ranges. This can be achieved by dividing the column dataset into multiple sections where each section is assigned to a processing unit. These units can then work simultaneously to determine the trading range or price change for each section and the results are then aggregated to produce the final output. This method is more efficient than the traditional approach as multiple

units can work on different sections of the same file concurrently, unlike the traditional method which processes data sequentially. It is important to note that the aggregation algorithm must be designed carefully to ensure the accuracy of results, especially with regards to time ranges in problem 4 as these errors in the time range could lead to inaccurate interpretations of the data.

5.5.4 Price Change & Trading Range

```
public class TradingRange {
    public static class RangeMapper extends Mapper<LongWritable, Text, Text, DoubleWritable> {
        private DoubleWritable tradingRange = new DoubleWritable();
        private Text date = new Text();
        @Override
        public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
            String[] columns = value.toString().split(",");
            if (columns.length >= 5 && !columns[0].equals("Date")) {
                String dateColumn = columns[0];
                String highColumn = columns[2];
                String lowColumn = columns[3];
                double highValue = Double.parseDouble(highColumn);
                double lowValue = Double.parseDouble(lowColumn);
                double range = highValue - lowValue;
                date.set(dateColumn);
                tradingRange.set(range);
                context.write(date, tradingRange);
            }
        }
    }
}
```

To begin with, I created Java files for TradingRange.java and Pricechange.java. In each file, I established a mapper that takes a LongWritable and a Text as input and outputs a Text and a DoubleWritable, representing the date and the price change/trading range respectively. A Text variable named "date" was then created to store the stock date. Subsequently, I implemented the map method for both classes. Within this method, I split the current line ("value") at each comma and stored it in an array of Strings called columns. An if statement was used to check that the number of columns is at least 5 and that the first column is not "Date". If this condition was met, the date, opening price and closing price were extracted and stored in String variables in the case of PriceChange, while for TradingRange, it was the date, high and low columns being extracted. The extracted values were then parsed into doubles. For TradingRange, the trading range was calculated by subtracting the high value from the low value, while for PriceChange, the price change was calculated by subtracting the open value from the close value. The date variable was set to the date column and the price change/trading range variable was set to the calculated change/range, respectively. Finally, the date and price change were emitted as output key-value pairs for PriceChange and for TradingRange, it was the date and trading range that were emitted.

```
public static class RangeReducer extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {
    @Override
    public void reduce(Text key, Iterable<DoubleWritable> values, Context context) throws IOException, InterruptedException {
        String range = key.toString().replace("Trading Range for: ", " ");
        for (DoubleWritable value : values) {
            context.write(new Text ("Trading Range for: " + range), value);
        }
    }
}
```

During the Reduce phase, I began by defining the Reducer class for TradingRange/PriceChange as

RangeReducer/PriceChangeReducer respectively. These classes accept a Text and DoubleWritable to represent the date and price change/trading range. Next, I implemented the reduce method. Within this method, I utilised a String variable called change/range to store the key, replacing the prefix "Price Change for: " / "Trading Range for: " to avoid duplication in the output file (e.g. Price Change for Price Change for ...). Lastly, I iterated through all values for each date, emitting a new key-value pair with the date prefixed with "Price Change for: " and the corresponding price change/trading range as the value.

5.5.5 Volume Rate of Change

```
public class VolumeChange {
    public static class VolumeChangeMapper extends Mapper<LongWritable, Text, Text, DoubleWritable> {
        private DoubleWritable volumeChange = new DoubleWritable();
        private Text date = new Text();
        private double previousVolume = 0.0;
        @Override
        public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
            String[] columns = value.toString().split(",");
            if (columns.length >= 7 && !columns[0].equals("Date")) {
                String dateColumn = columns[0];
                double volume = Double.parseDouble(columns[6]);
                if (previousVolume != 0.0) {
                    double volumechange = (((volume - previousVolume) / previousVolume) * 100.0);
                    date.set(dateColumn);
                    volumeChange.set(volumechange);
                    context.write(date, volumeChange);
                }
                previousVolume = volume;
            }
        }
    }
}
```

I began by creating a Java class named VolumeChange.java to compute the daily volume rate of change. Within this class, I established a mapper called 'VolumeChangeMapper' which takes a LongWritable and Text as input and produces a Text and a DoubleWritable as key-value pairs (representing the date and the volume change percentage, respectively). In the Mapper class, I declared a DoubleWritable variable named volumeChange to hold the volume change, a Text variable named date to store dates and a double variable named previousVolume initialized to 0 to hold the volume from the previous record. I then implemented the map method to process each record in the input data. I separated the current line's values by comma and stored them in an array of Strings called columns. I included a condition to ensure that there are at least 7 columns and that the first column is not "Date". If these conditions are met, the date from the first column is saved in a String variable and the volume is extracted from column 6 and converted to a double. If the previous volume is not 0.0 (to avoid calculating volume change for the first record), I computed the volume rate of change using the following formula:

$$VROC = \frac{V - pV}{pV} * 100$$

the previous volume (represented as pV) and current volume (represented as V). Following this step, the date and volume change values are assigned to the date and volumeChange objects respectively and the mapper outputs the date as the primary key and the volume change as the value using the context.write() method. Subsequently, the previousVolume variable is adjusted for the subsequent iteration by assigning it the value of the current volume.

```

public static class VolumeChangeReducer extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {
    @Override
    public void reduce(Text key, Iterable<DoubleWritable> values, Context context) throws IOException, InterruptedException {
        String volumeRateChange = key.toString().replace("Volume Rate of Change for:", " ");
        for (DoubleWritable value : values) {
            context.write(new Text ("Volume Rate of Change for:" + volumeRateChange), value);
        }
    }
}

```

Next, I created the Reducer class for VolumeChange.java, naming it VolumeChangeReducer. Within this class, I utilised the reduce method, which is called once for each key (date) and its list of associated values (volume changes). I identified an issue with the duplication of a specific prefix, "Volume Rate of Change for:", in the final output file. To address this, I converted the prefix to a String and replaced it with spaces. I then iterated through the list of DoubleWritable values associated with the key. For each value, I emitted key-value pairs where the key consisted of the modified variable VolumeRateChange with the text "Volume Rate of Change for:" appended to it and the value remained unchanged (representing the volume change percentage for the date).

5.5.6 RSI

```

public class RSI {
    public static class RSIMapper extends Mapper<LongWritable, Text, Text, DoubleWritable> {
        private ArrayList<Double> closePrices = new ArrayList<>();

```

I began by creating a class for the Relative Strength Index (RSI) in which I will be executing RSI calculations for A.csv. Within this class, I expanded the native Hadoop mapper and named it RSIMapper. The RSIMapper class takes a LongWritable and a Text as input and outputs a Text and a DoubleWritable, representing the date and calculated RSI, respectively. Additionally, I established an ArrayList named closePrices to monitor the closing prices of the financial data.

```

@Override
public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
    String[] columns = value.toString().split(",");
    if (columns.length >= 7 && !columns[0].equals("Date")) {
        double closePrice = Double.parseDouble(columns[4]);
        closePrices.add(closePrice);
        if (closePrices.size() > 30) {
            double upwards = 0.0;
            double downwards = 0.0;
            for (int i = 1; i < closePrices.size(); i++) {
                double difference = closePrices.get(i) - closePrices.get(i - 1);
                if (difference > 0) {
                    upwards += difference;
                } else {
                    downwards += 0 - difference;
                }
            }
            double averageUpwards = upwards/30;
            double averageDownwards = downwards/30;
            double relativestrength;
            if (averageDownwards == 0) {
                relativestrength = 99.0;
            } else {
                relativestrength = averageUpwards/averageDownwards;
            }
            double rsi = 100-(100/(1 + relativestrength));
            context.write(new Text(columns[0]), new DoubleWritable(rsi));
            closePrices.remove(0);
        }
    }
}

```

Next, I created a map method to iterate through each entry of the dataset. Within this method, I split the input Text data using a delimiter on the comma and convert values to strings, storing them in an

array called columns. If there are at least 7 columns and the first column is not "Date", then I am able to process the data without encountering an array out-of-bounds error. I then extract the closing price from column 5 of the dataset and store it in a variable named closePrice, which is added to an arraylist called closePrices. If there are more than 30 closing prices available, I initialize two variables, upwards and downwards, to accumulate positive and negative differences between consecutive closing prices. I calculate the average upward and downward movements by iterating through the closePrices list, finding the difference between consecutive elements and dividing the sum by 30. A variable called relativeStrength is declared to calculate the RS (part of the RSI formula) later on. The relative strength ratio is then calculated as the average upward movement divided by the average downward movement, with the relative strength being set to 99 if the average downward movement is zero.

Finally, I calculated the RSI using the following formula:

$$RSI = \frac{100}{1 + \frac{n_{up}}{n_{down}}}$$

The date and RSI are then outputted as key-value pairs by the mapper. But, in order to keep a window of the last 30 closing prices, the oldest closing price is then removed from the list of closing prices.

```
public static class RSIReducer extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {
    @Override
    public void reduce(Text key, Iterable<DoubleWritable> values, Context context) throws IOException, InterruptedException {
        String RSImetric = key.toString().replace("RSI for:", " ");
        for (DoubleWritable value : values) {
            context.write(new Text ("RSI for:" + RSImetric), value);
        }
    }
}
```

In the Reducer class, I extended the native Hadoop Reducer class and named it RSIReducer. Within this class, I defined the reduce method which iterates through all key-value pairs emitted by the mapper to combine values associated with the same key. In this method, I created a String variable named RSImetric to extract the key as a string and remove the text "RSI for:" to address duplication issues. Additionally, for each key, I outputted a new key-value pair with "RSI for" concatenated with the date and the RSI value.

5.5.7 Interpretation

As nearly all the the metrics I have analysed are associated with the stock A.csv, I intend to focus my discussion on the interpretations and findings of the results specifically in relation to this company. A.csv pertains to the stock price data of Agilent Technologies Inc., a company that specializes in providing instruments, software and services for laboratories. Agilent Technologies Inc. transitioned into a publicly traded company in November 1999 (24) following its Initial Public Offering (IPO) at a share price of \$30. The company was initially a spin-off from Hewlett-Packard (HP) and entered the stock market during a period of market uncertainty, characterized by the dot-com bubble and subsequent recession, resulting in significant fluctuations in its stock value. Over the past few years, Agilent has experienced consistent growth and expansion thanks to its diverse portfolio of products and services catered to industries including pharmaceuticals, biotechnology and environmental testing. Agilent's emphasis on enhancing shareholder value through dividends and share buyback initiatives showcases their commitment to implementing strategic capital allocation strategies that benefit shareholders and drive company growth. Indeed, when analysing the price fluctuations of Agilent's stock, it is evident that there was a period of significant volatility in the early 2000s, attributed to the dot com bubble. Being a technology-focused company, Agilent was particularly affected during this time. There was a drastic drop of over \$32 in its stock value in a

single day, followed by a partial recovery. Additionally, from 2003 to 2006, there was a relatively stable period with minimal price changes, although there were spikes in 2005 due to KKR and Silver Lake Partners acquiring Agilent's Semiconductor Products Group for \$2.66 billion and in 2007 (25) when Agilent purchased Stratagene to enhance its laboratory automation and informatics capabilities. This period of modest growth can be attributed to reduced investor confidence in the technology sector, resulting in a more cautious approach to technology investments and low demand for computer equipment and software. It is worth noting that, unlike many other technology companies, Agilent did not experience significant downturns during the recession of 2007, likely due to its diversified business model and focus on laboratory instruments/services essential for scientific analysis. Agilent's emphasis on serving industries with consistent demand, such as healthcare, diagnostics and genomics, provided a level of resilience against economic fluctuations. Additionally, the US government did not reduce research expenditure during the 2007 recession (26) and actually increased discretionary spending to stimulate the economy. While Agilent did face some minor setbacks during the recession, it was able to recover quickly. When looking at the historical trading patterns of Agilent, it is apparent that there were two significant periods of fluctuation in its stock value. Specifically, these occurred during the early 2000s and after 2018. The volatility witnessed in the early 2000s can be linked to the impact of the dot com bubble on technology companies, resulting in a substantial decline in Agilent's stock price from \$80.83 to \$9.57. More recently, in the 2018s, there was another period of increased volatility with daily price fluctuations ranging from \$3-\$5. This growth can be attributed to various financial factors such as strong financial performance, favorable earnings reports and positive analyst upgrades. For example, in the fourth quarter of fiscal year 2018, Agilent reported a revenue of \$1.29 billion, showcasing a 9% growth rate with core revenue growth also at 9%. The annual revenue for the company was \$4.91 billion, showing a growth of 10% overall and a core growth of 7%. This positive performance likely contributed to increased investor confidence and improved stock performance. In 2018, 14 analysts provided ratings for Agilent Technologies, with 10 recommending a buy or overweight rating, 3 recommending a hold and 1 recommending a sell or underweight rating. The majority of analysts recommending a buy likely boosted investor confidence in the company. During the COVID-19 pandemic, Agilent experienced strong performance due to the high demand for laboratory equipment and high throughput processing for virus studies. The company's revenue increased by 3% in fiscal year 2020 compared to 2019, reaching an all-time high stock price of \$179.28 in August 2021. The increased global focus on healthcare and research during the pandemic resulted in higher spending in these areas, benefiting companies like Agilent that supply essential tools and equipment for research and development. This period of growth can be observed in Agilent's trading range, with the company's stock experiencing regular daily price fluctuations of around \$7 towards the end of 2020, often trending upwards. Another important metric to consider for Agilent is the Volume Rate of Change (VROC), which is utilised to validate the direction of price trends. Analysis shows that during the dot com bubble, there were significant volume changes, reflecting a bearish sentiment among investors who rushed to sell shares due to Agilent's presence in the technology sector. Furthermore, there was a noticeable increase in trading volume in 2013 following the release of fourth-quarter results, a trend that continued post-2018 with the release of positive earnings reports, boosting investor confidence and leading to an upward trend in stock prices. Conversely, during the great recession, there was a decrease in VROC signifying diminished interest from sellers, resulting in the downtrend losing momentum. Observing the graph, it becomes apparent that in the 2020s, the trading range, price change and volume rate of change have all shown improvement, indicating growing investor confidence and potentially positive performance for Agilent's stock. Then there is the Relative Strength Index (RSI), which shows that between 2006-2007, the stock did not reach extreme overbought or oversold levels, indicating a period of fluctuation and relative stability. Additionally, post-2017, Agilent's stock did not become heavily oversold, suggesting a gradual decline rather than a drastic drop. This steady performance can be attributed to the company's strong financials and positive market sentiment. Ultimately, by utilising key metrics such as price change, trading range, volume rate of change and the RSI, one can extract valuable insights from the company's stock and analyse the findings. While using only these 4 indicators may not be sufficient for making accurate future predictions, they do provide a foundation for understanding historical data and identifying trends in the stock. For example, towards the end of 2020, Agilent exhibited significant fluctuations in the trading range and price change along with an increased volume rate of change and some overbought

signals in the RSI. These signals indicated fast upward movement in the stock price, which aligned with the high demand for laboratory equipment during the COVID-19 pandemic, which resulted in substantial growth for the company.

5.5.8 Python Files

Given that the Python files responsible for generating the graphs for each indicator in problem 4 share a high degree of similarity, with differences primarily in the graph design and input information, I will focus on discussing the core structure of these Python files where the logic for graph generation remains consistent.

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from datetime import datetime
```

I began the process by importing the matplotlib plotting library into Python. I utilised `matplotlib.use("Agg")` to generate plots from the Linux terminal. Next, I imported the pyplot module from matplotlib and assigned it the alias "plt". Additionally, I imported the datetime module from the datetime library to manage dates related to stocks.

```
dates = []
rsi = []
with open("RSIoutput/part-r-00000", "r") as file:
    for line in file:
        if line.startswith("RSI for: "):
            parts = line.strip().split()
            date = parts[2]
            rsidata = float(parts[3])
            date = datetime.strptime(date, "%Y-%m-%d")
            dates.append(date)
            rsi.append(rsidata)
```

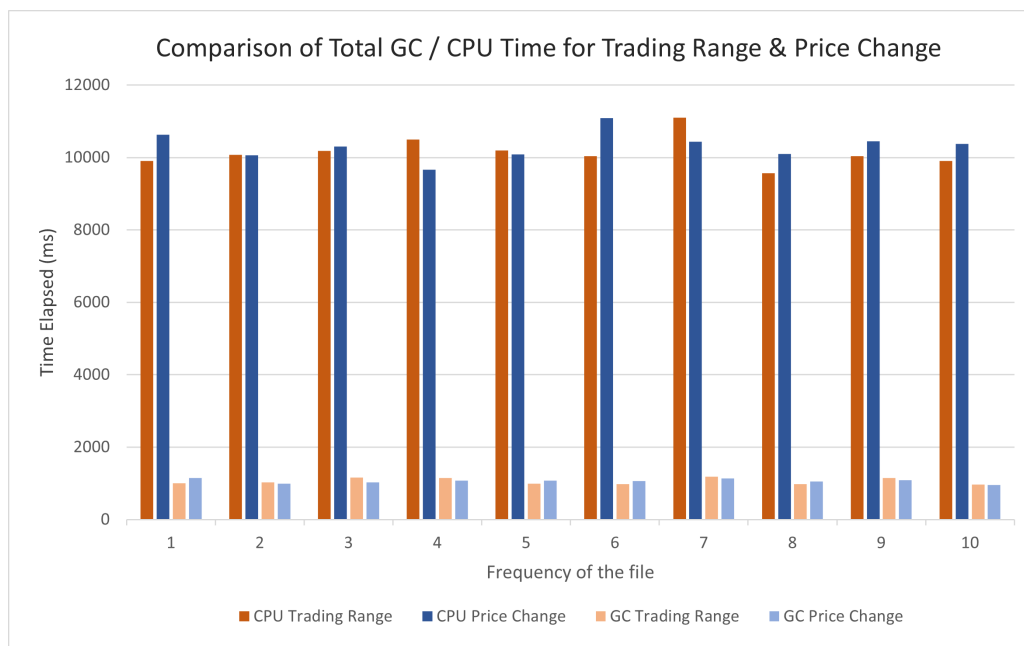
Next, I initialized two empty lists named `dates` and `<variable_name>` to store the dates and values of the variable I am calculating respectively. I then opened the file named `part-r-0000` (output file from a Hadoop job) in a specific directory in read mode using the `open()` function. I iterated through each line in the file using a 'for' loop. If a line begins with "`<variable_name> for:` ", I extracted the date and `<variable_name>` value from the line using the `'strip()'` and `'split()'` functions. These functions are similar to `trim()` and `split()` methods in Java, respectively. The extracted information was stored in a list called `parts`. I determined the position of the date and `<variable_name>` value within the list by analysing the output line structure. Next, I converted a string representing a date in the format of year-month-day into a datetime object using `datetime.strptime()`. Finally, I added the extracted date and `<variable_name>` value to their respective lists.

```
plt.figure(figsize=(12, 7))
plt.plot(dates, rsi, color="#2a2a2a", marker="o", linestyle="-")
plt.title("RSI for this company stock")
plt.xlabel("Date")
plt.ylabel("RSI")
plt.yticks(range(0, 105, 5))
plt.axhline(70, color="red", linestyle="--", linewidth=3)
plt.axhline(30, color="green", linestyle="--", linewidth=3)
plt.grid(True)
plt.savefig("RSI.png")
```

In the end, I generated a new figure of a specific size using `plt.figure`. I then proceeded to represent the data from the dates and `<variable_name>` lists with a solid line, circular marker and a dark gray color (`#2a2a2a`). Subsequently, I provide a title for the graph and label the x and y axes. I adjust the y-axis to display a range of values with a set interval between them, a process that involved some experimentation. Additionally, for the RSI data, I include two extra lines at 30 and 70 to signify overbought or oversold conditions. Lastly, I enabled the gridlines on the graph and saved it as a PNG image in the current directory.

5.5.9 Performance Analysis

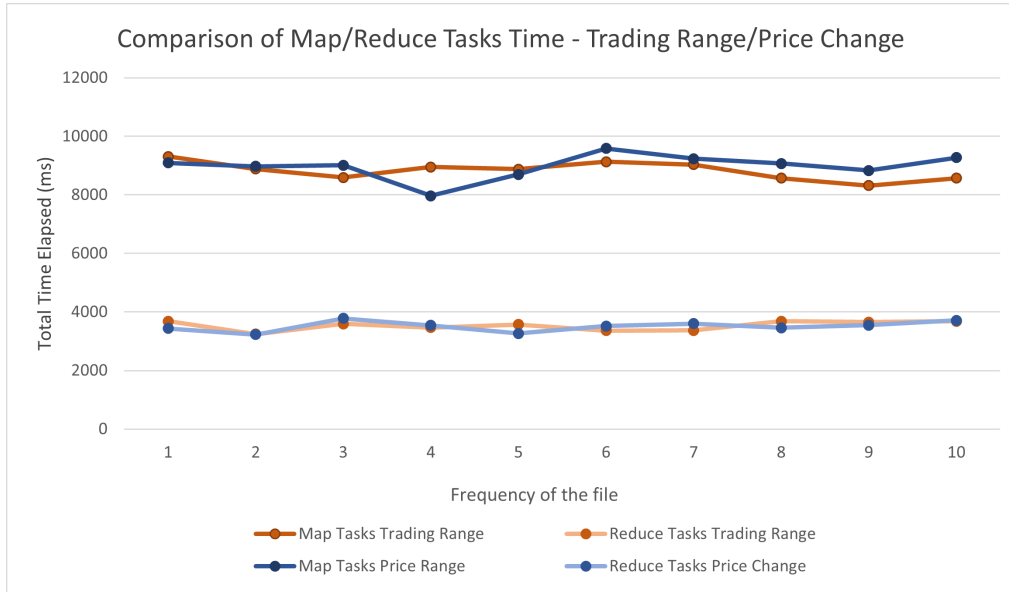
One important point to note is that I will not be addressing the performance analysis of other Java files as they exhibit very similar performance statistics. Moreover, discussing these files would not provide any new insights as the minimums and peaks in map/task times or CPU/GC times are consistent across all Java files for problem 4. Therefore, I will focus solely on the performance of `TradingRange.java` and `PriceChange.java` as an example.



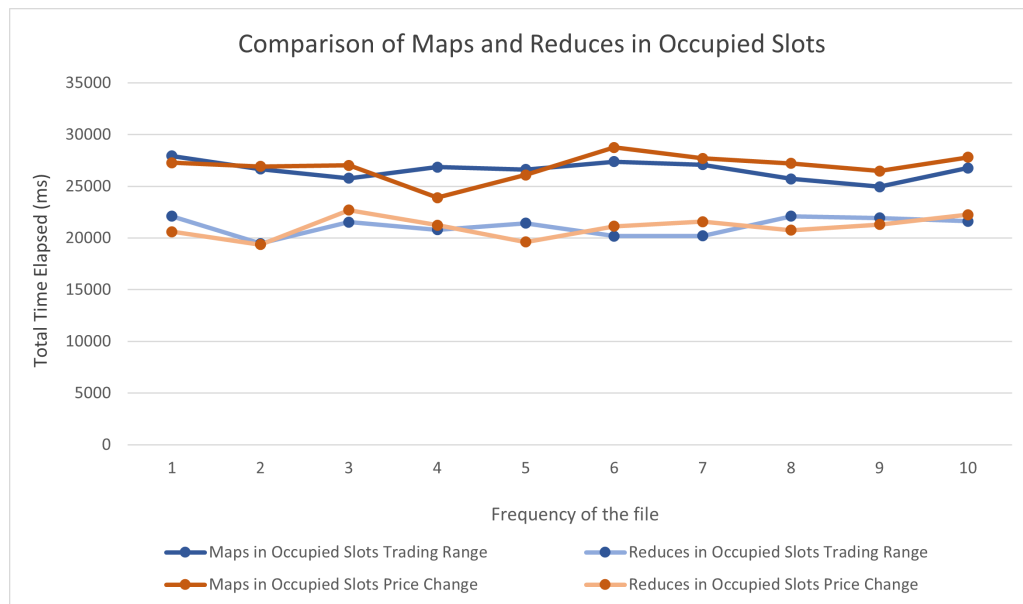
An interesting observation is that despite both Java files exhibiting peaks and minimums at similar times, there is a difference in the frequency of the dataset file at which these times occur between the two files. Indeed, while the peaks occur at similar dataset frequencies (7 & 6), the minimums occur at 8 & 4 respectively. Given that both files contain algorithms of similar complexity (linear), it might be that environmental factors such as system load and resource contention in the cluster may have contributed to these results.

On the other hand, the garbage collection time remains relatively consistent across multiple dataset

files during the execution of tradingrange.java and PriceChange.java, where it does not exhibit significant peaks or minimums. This can be observed by analysing the mean and median for tradingrange.java, which are 1057.8 and 1015 respectively (which are somewhat conformed). Moreover, with a standard deviation of 88.06 and a correlation coefficient of -0.06, it is evident that the GC time values exhibit some level of invariability (as the standard deviation is greater than the absolute value of the difference between mean and median). Similarly, PriceChange.java displays a mean and median of 1060.4 and 1070.5 with a standard deviation of 58.63 and a correlation coefficient of -0.197. This suggests that the values for pricechange also remain relatively constant, albeit with slightly more variability compared to tradingrange.java. Overall, these results indicate that resource utilization within the system remains consistent for both java files regardless of the number of datasets being processed.



In regards to the timing of map/reduce tasks, the time for Map Tasks for TradingRange.java generally stays consistent, although it slightly decreases as the number of dataset files processed increases (with a negative correlation coefficient of $r = -0.61923$). On the other hand, the time for map tasks for PriceChange shows a notable low point at 4 dataset files processed (7970ms) and a peak at 6 files processed (9585ms). This suggests that the java file PriceChange may have a lighter workload when handling 4 csv datasets (possibly due to parallel processing) or it might be an outlier case due to environmental factors. However, the reduce task times for both trading range and price range exhibit a relatively constant trend with correlation coefficients of 0.32 and 0.375 respectively. This indicates that despite there being some variation in map task times, reduce task times tend to be more consistent and unchanged.



In the case involving map/reduce task durations in occupied slots, which refers to the time taken for individual map and reduce tasks to run on available computing resources such as nodes, the map durations for both the TradingRange.java and PriceChange.java files show some variability due to their high standard deviation of 875.6 ms and 1288.82 ms for TradingRange.java and PriceChange.java respectively. Although the map task durations in occupied slots are fairly consistent for tradingrange.java, there is a noticeable peak at 6 CSV (28755 ms) datasets processed and a low point at 4 CSV (23910 ms) datasets processed for PriceChange.java. This suggests that as observed in the map/reduce task durations, some external factors such as the current usage of computing resources within the Hadoop cluster may have influenced this variation. On the other hand, the reduce tasks for both java files remain relatively stable with no significant fluctuations.

In the end, the peak at processing 6 CSV datasets for PriceChange.java could have been influenced by external factors rather than it being the most efficient workload for parallel processing in the Hadoop cluster. This is because the mapper/reducer algorithms are quite similar in both Java files, as well as the computations and the data being processed. Indeed, if 6 CSV datasets were an optimal workload for parallel processing in the cluster, TradingRange.java may have experienced a similar minimum in processing time as well.

6 Critical analysis and Discussion

6.1 Reflections

As the term concludes, I find myself reflecting on my accomplishments and the valuable lessons I have learned during this journey.

Indeed, I must admit that there are areas where I could have improved such as in time management. In fact, due to the workload of my current joint-degree course, I had to deal with multiple assignments each week which made it challenging to find a balance between my project and any other modules, hence making slower progress. Nonetheless, I recognise that if I improved my time management and planned all of my study sessions, I would have sped up the progress on my project.

Nonetheless, I successfully completed all tasks and my report and I gained a comprehensive understanding of Hadoop, big data, parallel processing and other related concepts that may benefit me in the future.

Overall, I am pleased with my project, though I believe there is room for improvement in the structure of my report.

In the end, I have gained substantial knowledge and experience during this project and I look forward to utilise it in my professional ambitions.

6.2 Difficulties

During the this year, I encountered significant challenges that have both tested and strengthened my knowledge of Hadoop. These challenges include.

- **Communication:** One significant challenge I encountered this term was unclear communication with my supervisor. In the first week of Term 1, a misunderstanding led me to use Hashmaps instead of the usual Hadoop map/reduce methods. Additionally, my supervisor's once suggested me the use of two mappers for problem 2 (to retrieve the line number associated with the matched String) after he misunderstood the scope of problem 2, which resulted in unnecessary effort and wasted time. Indeed, clear communication could have prevented these issues, hence, potentially saving me two productive weeks and providing me with a starting point for addressing problem 3 during Term 1.
- **Wasted time:** During term 1, I spent around two weeks lingering around the issue of calculating the line number for a matched string in problem 2 and despite multiple attempts, I couldn't resolve it until my supervisor highlighted during one of my FYP sessions that the `byteOffset` in the input key of the `TextInputFormat` could be used to derive the line number. In fact, the day after this interaction, I managed to fully complete problem 2. Indeed, if I had conducted further research beforehand on certain aspects of MapReduce, such as `FileInputFormat`, I could have avoided the two week delay, which would have provided me with additional time to address problem 3.
- **Configuration Issues:** Understanding the Hadoop configuration and its terminal commands proved challenging to me as I was new to running applications via commands. In fact, understanding and using the appropriate commands took considerable time, which I was able to accomplish only after seeking guidance from my supervisor and other students. Moreover, executing a Java file in Hadoop required around 6 commands, which made the debugging process quite cumbersome. Furthermore, Hadoop's requires a significant amount of time to process the input data and obtain the final output, which made it challenging to verify any issues present in the output.
- **Insufficient Resources:** The limited availability of resources for certain Hadoop/MapReduce aspects posed a challenge for me to troubleshoot any issues present in my code or to comprehend certain aspects of the syntax. Moreover, specific errors in the terminal lacked resources/help online, which I only resolved after seeking advice from my supervisor.
- **Time Management:** During both term 1 and term 2, time management proved challenging due to the demands of my joint-degree course. In fact, balancing the workload between the project and my frequent maths assignments became increasingly difficult, particularly towards the end of each term when multiple modules scheduled their assignments deadlines towards the end of the term. Consequently, I found myself working sporadically on the project and trying to cram tasks just before my FYP meetings with my supervisor. This situation was particularly challenging towards the end of term 2 when I had to complete the report. Due to finishing problem 4 a bit behind schedule, I was forced to request an extension. Unfortunately, this circumstance resulted in the loss of valuable study time that I needed for preparing for the exams.
- **MapReduce Paradigm:** Understanding the Hadoop MapReduce paradigm in Term 1 presented a steep learning curve which required a significant amount of time and effort to grasp. Indeed, the complexities of dividing any problems into "map" and "reduce" phases to optimise data processing efficiency posed a challenge.

- **Algorithm Design for Problem 3:** Given that problems 1 and 2 primarily required simple calculations, problem 3 caught me off guard as it required me calculating various order statistics such as the mean, mode, maximum, etc. within a single file. Initially, I was unaware of the ability to utilise the `method context.write()` multiple times, which lead to some challenges with this problem. However, to streamline the debugging process, I decided to calculate each statistic individually in separate Java files and consolidate the results afterwards.
- **File Aggregation:** For problem 3, after completing the calculations of the order statistics, I encountered challenges during the aggregation process. Specifically, aggregating the logic for calculating the mode with the logic for the other calculations proved to be the most complex aspect. This difficulty arose from the fact that the format of key-value pairs emitted by the mapper was different, where the key represented the mass excess uncertainty value and the value represented a count (1.0). Ultimately, I was able to resolve this issue using Hashmaps, however, despite this solution working, it is not optimal for Hadoop environments as Hashmaps can be inefficient and consume a significant amount of internal memory in the Hadoop cluster.
- **Cluster Shutdowns:** During certain periods throughout the day (mostly during nights) or when working with multiple datasets, the hadoop cluster would occasionally experience shutdowns and give me errors when accessing it, which prevented me from making commits or debugging files. This presented a challenge towards the end of term 2, as the completion of my code on time was crucial for me to begin working on the report.

6.3 Successes

Throughout this year, I achieved numerous successes with respect to this project, which include:

- **Project Completion:** By the end of term 2, I successfully completed my final year project and addressed all four problems outlined in the project list. Due to my initial unfamiliarity with Hadoop and my limited experience in data analysis, I anticipated this project to be challenging. While I did encounter difficulties along the way, I was able to overcome them through independent research and discussions with my supervisor. This allowed me to develop a strong understanding of Hadoop and successfully address all four problems.
- **Professionalism:** When working on this project, I gained valuable insights on how to handle a project in a professional manner. Indeed, through utilising tools like git, keeping a diary, writing the reports and attending supervisor meetings, I developed a clear understanding of the professional approach to tackling a computer science project. Additionally, as this project was to be completed independently, I learned essential skills such as research skills, self-discipline and time management.
- **Hadoop MapReduce:** Through this project, I was able to develop a strong understanding of the Hadoop ecosystem, primarily the HDFS and MapReduce. I also gained knowledge about mapping, shuffling and reducing data in distributed computing environments, which could be beneficial for me in case I used any similar frameworks in the future. Moreover, this experience has been particularly valuable to me as I have a keen interest in big data and have learned how to effectively utilise frameworks like Hadoop for processing large datasets, which will be advantageous to me in industry.
- **Big Data Analysis:** While the datasets utilised can not be classified as big data, this project provided me with valuable experience in conducting basic data analysis and deriving insights from my findings. Problem 3 and problem 4 were particularly interesting in this aspect, as I was able to improve my understanding of various concepts through data interpretation. For instance, in problem 4, I found it intriguing to analyse stock fluctuations within the data and verify the underlying reasons for this occurrence.

Regular Expressions:

- Throughout this project, I frequently utilised regex expressions which helped me gain an understanding of their application in various scenarios. For instance, in addressing problem 1, I utilised regex expressions to extract data from a specific section of the HTML file. On the other hand, in addressing problem 2, I utilised regex expressions to identify and extract strings within the input text file that matched the regex expression in question.
- Parallel Computing: During this project, I gained knowledge of the principles of parallel computing and various techniques used to process and analyse large datasets. While I did not modify any cluster configurations, I gained insight into parallel processing by conducting performance analysis on problem 3 and problem 4. This analysis helped me determine the optimal number of files to be processed by the cluster at Royal Holloway, as well as identify how environmental factors can impact the duration of map/reduce tasks (in the case of `TradingRange.java/PriceChange.java`).
- Data Visualisation: During this project, I acquired the skill of visualising data using matplotlib in Python. While I borrowed a code snippet (27) for calculating the bar duration in problem 3, I was able to grasp the concept of presenting statistical findings and inferring from the results. Additionally, I learned how to utilise the Linux command line for generating the graphs in python, which is a technique that is not commonly used.
- Problem Solving: Upon reflecting on my project, I was able to improve my problem solving skills by effectively addressing each of the problems outlined in the final project list. In particular, in addressing problem 3, I was initially uncertain on how to approach the task of calculating each statistic within a single Java file. However, I later chose to calculate the statistics separately and then aggregate the logic at the end. This approach not only saved me a significant amount of time but also streamlined the debugging process.

6.4 Future Enhancements

If I were to start this project again, I would focus on improving my time management in order to efficiently handle the coding and the report. In addition, I would have tried to improve my communication with my supervisor to avoid any misunderstandings which ultimately lead to wasted time. Moreover, I would explore new features such as enabling my programs to process multiple datasets as I attempted with problem 4, and implementing algorithms to gain insights from data, potentially utilising machine learning techniques in the process. Additionally, I would work on improving my communication skills for presentations, as I was somewhat nervous and lacked experience speaking in public. Nevertheless, I would still choose a topic related to data analysis with Hadoop as I found it to be highly enjoyable.

7 Professional Issues

7.1 Hadoop

Initially, Hadoop lacked robust encryption features and some authentication mechanisms, which posed significant risks for data privacy and integrity. Moreover, Hadoop is also not secured against data breaches as its framework is written in Java, which is vulnerable to cyber-attacks and has been exploited by cybercriminals in numerous cases in the past.

However, subsequent versions included improvements such as end-to-end encryption and compatibility with Kerberos-based authentication which effectively mitigated some of these vulnerabilities. Furthermore, improvements in user authentication, such as integration with Active Directory/LDAP solutions and

the use of access control mechanisms such as Ranger/Sentry have enhanced Hadoop's security and enabled secure user access. However, despite these enhancements, challenges still exist in managing encryption keys and implementing data masking.

The security vulnerabilities in Hadoop could potentially expose personal data stored in the HDFS, leading to unauthorized access to sensitive information. This poses a risk to individual privacy and can also put an organization's confidential data at risk. To address this issue, it is crucial to implement encryption methods such as Transparent Data Encryption (TDE). TDE encrypts data at rest in the HDFS, ensuring that data blocks are encrypted before they are written to the disk and decrypted only when accessed.

7.2 Big Data

Big data, which is characterised by vast volumes and diverse sources of data, necessitates ethical considerations such as data ownership, privacy, consent, openness, etc. which are crucial for maintaining trust and fairness in the use of data across various industries.

- **Ownership:** Determining ownership of data involves the establishment of rights over property, which includes control and limitations on sharing personal data that contributes to an individual's digital identity. This issue becomes apparent when individuals document observations about others, which results in conflicting claims over the data and potential consequences for one's digital identity. Especially in today's digital age, the extensive collection and monitoring of individuals behaviours online highlight ethical considerations regarding privacy rights and intellectual property.
- **Transparency:** Big data ethics includes concerns about controlling the predictions made using algorithms, particularly when utilising predictive analytics to anticipate future behaviors as its outcomes can inadvertently perpetuate disparities, particularly when data sets predominantly represent certain demographic groups. Notably, the homogeneity of certain data sets, such as genetics data, poses significant concerns regarding its applicability and exacerbation of existing gaps in medical knowledge.
- **Privacy:** In Big Data, privacy raises ethical dilemmas (28) revolving around the balance between data usage and privacy rights. While data sharing holds potential benefits for tailored analyses and insights, it also raises apprehensions about the loss of control over personal information and potential exploitation. This critical importance of privacy stems from its role in limiting government and corporate power, which empowers individuals to control their lives, decisions and protecting them from exploitation. However, the advent of Big Data research introduces significant challenges to privacy, particularly evident in the potential for privacy violations. The capability of Big Data to swiftly transition into mass surveillance raises profound concerns (29), as demonstrated in various scenarios where algorithms are deployed for intrusive purposes, such as assessing risk in employment processes without individuals consent or awareness. Furthermore, the use of de-identified data, often presumed to safeguard privacy, faces mounting risks of re-identification, as illustrated by studies involving genetic data. The ease with which de-identified data can be linked back to individuals via publicly available information underscores the need for enhanced guidelines and protections in Big Data research.
- **Worth:** Furthermore, ethical dilemmas emerge when individuals are insufficiently informed about the collection, storage and usage of their data, leading to feelings of violation and mistrust due to a lack of control. The economic value of data raises concerns regarding fair compensation for individuals contributing to data collection, highlighting disparities between the value derived from data-driven services and the equitable compensation for data usage highlights issues of transparency and equity in data exchange.
- **Openness:** The concept of open data revolves around the notion that information should be easily accessible without unnecessary limitations such as copyright laws. Indeed, Scholars argue

that open data fosters societal democratization by allowing broad participation. Despite varying attitudes toward data sharing, the democratising potential of open data remains a cornerstone of ethical considerations in the realm of big data

Data breaches pose significant threats to individuals, potentially resulting in severe consequences such as identity theft and financial loss. Since organizations hold sensitive data, they bear the ethical obligation (30) to safeguard data through robust cybersecurity measures. Implementing stringent security protocols not only protects individuals' privacy and financial well-being but also upholds trust and integrity in the organization's operations.

7.3 Final Year Project

During my final year project, it was crucial for me to exercise caution when obtaining datasets for problem 3 and 4. It was necessary for me to consider any ownership rights associated with the data I used in my project. For problem 3, I acquired a dataset from the National Nuclear Data Center (NNDC), which provides access to their public databases for general use. For problem 4, I obtained a dataset on NASDAQ stock/etf data from kaggle. I ensured to respect the intellectual property rights of the organizations that granted me access to their data. Additionally, when utilising code from third parties found online, such as the snippet used in the python script to generate a frequency distribution graph for problem 3, I always acknowledged the source within the file and in my report. Additionally, when working with collections of web pages or datasets, it is essential to prioritize data confidentiality and prevent the spread of personal information to uphold individual privacy. For example, in addressing issue 1, while examining a collection of web pages related to poetry, it was noted that many poets had included personal information in their works. It was imperative to handle this information with care and not include such details in the analysis conducted for issue 1, which involved counting word occurrences in multiple files. In addition, it was crucial for me to avoid bias in my analysis and to be fully transparent in my data interpretation. Specifically, when addressing issue 3, I worked with a nuclear decay dataset that included experimental data susceptible to misinterpretation. It was imperative for me to approach the interpretation of certain dataset features with caution, as many variables were new to the scientific community. Given my lack of professional background in particle physics, it was essential for me to avoid making unfounded extrapolations. Any biased analysis on my part could potentially be viewed as misinformation from a technical standpoint, which has the potential to cause significant harm if taken seriously by even a small subset of individuals. Due to Hadoop's limited encryption capabilities, there is a possibility of encountering challenges when storing data in the Hadoop Distributed File System (HDFS). The HDFS does not have native encryption at rest, which leaves the data vulnerable to unauthorized access if the clusters security is compromised. In the event that the Hadoop cluster at RHUL does not have transparent data encryption (TDE) or any other encryption measures in place, there is a risk of exposing downloaded datasets, work and personal information. Unauthorized access to the HDFS could potentially lead to the exposure of personal information contained within java files, especially if Javadoc was used.

8 Appendix

8.1 Diary

Term 1

Week 1: I spent week 1 trying to understand how to use Hadoop and familiarising myself with its syntax. I borrowed a book from the library (Hadoop: the definitive guide) and watched a couple of YouTube videos in order to gain an understanding of the basics of Hadoop. Thereafter, to practise my knowledge, I tried unsuccessfully to setup Hadoop on my local system for which I suspect that

my environment variables were not setup the correct way. Therefore, I started using noMachine in order to familiarise myself with its usage since it easily allows you to connect to the big data system at RHUL. Moreover, I noticed that my git repository has not been created so I emailed the CIM helpdesk team to sort out the problem.

Week 2: I spent week 2 working on problem 1. During this week, all my efforts were dedicated to writing a java file such that it takes a text file as an input and outputs the number of occurrences for each word contained in that text file. Although I did not specify that a text file was going to be the input, I still continued with the idea since I received approval from my supervisor. The task itself was not difficult at all as I have tackled similar tasks in the past. On a side note, I used hashmaps over the traditional Mapper/Reducer classes offered by Hadoop, hence, I had to start again and implement what I implemented with Hashmaps, onto a new class with a Mapper and Reducer.

Week 3: During the start of week 3, I modified the code for problem 1 such that it now takes a few HTML pages (poetry related webpages) as input and then outputs the occurrence of each word in the poem. The task proved to be a bit more difficult than previously thought since I had to perform the counting of each word for only a section of the webpage (the poem itself). To tackle this problem, I used regex so that the algorithm is only performed when it encounters the div with ("id=post_description") and it stops after its closure. Moreover, I used regex in order to omit any words starting with "href" and any HTML tags from the word count.

Week 4: I spent the entirety of this week working on problem 2 and I managed to implement a basic distributed grep functionality that extracts each word that matches a specific regex expression ("in this case any word that contains h") and output the word alongside its line number. However, the line number is not accurate and there may be a logical error in the code. However, the basic functionality for the code is there.

Week 5: During this week, I tried to fix the error relating to the line number, albeit I was not successful. However, I made slight modifications to the code such that it now extract each word matching the regex in question from a section of the HTML page. Indeed, the section of the HTML page is enclosed by ("id=post_description") and a ("div") , which allows me to extract only the necessary information. Furthermore, I made slight changes to the WordCount.java file.

Week 6: I spent this week trying to figure out how to output the correct line number but after multiple attempts it outputs a slightly different number from what expected. I opted to use a while loop and iterate through each string that matched the regex expression in question, afterwards a variable named linenummer was incremented. However, this method does not seem to work. Moreover, I uploaded 5 more HTML pages containing more text to the repository, they are going to serve for testing.

Week 7: During a FYP meeting, my supervisor suggested the idea of using two mappers and chaining them together in order to output the current line number, however, after many efforts, I was unsuccessful. I researched this topic a lot and I tried to chain two mappers but it proved very complicated to implement for a program that implements a distributed grep functionality. I was unsuccessful at finding any resources online that could help me with this problem.

Week 8: During this week, I managed to finish problem 2 completely and I started working on my interim report. Indeed, during a meeting with my supervisor, it turned out that there was a misunderstanding and the idea of implementing 2 mappers and chaining them together was ineffective for task 2. Moreover, it turns out that the key input for FileInputFormat for a mapper represented the byteOffset of the line in question. Indeed, through some research online, I managed to implement "LineNumberInputFormat" which extends FileInputFormat such that instead of displaying the byteOffset, it now displays the actual line number. In order to implement this functionality, I borrowed sections of code online as this functionality was not inherently requested by my supervisor as well as for the fact that I did not have any knowledge regarding the class FileInputFormat.

Week 9: During this week, I started to exclusively dedicate all my efforts towards finishing off most of the sections in my report. In fact, I managed to finish certain sections such as the Abstract and

the entirety of Project Progression. Also, I started working on my presentation and I successfully managed to submit it before the deadline. Moreover, I submitted a couple of drafts to my supervisor and I received valuable feedback that helped me improve my report.

Week 10: During this week, most of my efforts were concentrated towards finishing up my report and preparing myself for my presentation. Indeed, I spent the first few days memorising a script to recite during my presentation, which ended up going Ok. I finished the interim report during this week.

Term 2

Week 1: During this week I started experimenting with the dataset "NuclearDecay.csv" and made some progress regarding the calculation of the mean for the variable "massExcessUncertainty". However, not much progress has been made since I was unsure how to calculate other variables from the same java file.

Week 2: During this week, I partially managed to calculate the mode, that is, I calculated the cumulative frequencies for each of the values in the "massExcessUncertainty" column and outputted them using mapReduce. However, calculating the mode proved to be a bit difficult as many calculation methods proved to be unsuccessful.

Week 3: During this week, I looked up resources on how to calculate the mode whilst utilising only the mapper and reducer classes but this proved to be quite difficult to implement in hadoop for my specific scenario. Due to this reason, I had trouble making any progress for this week.

Week 4: During this week, I looked up resources for ways in order to output multiple calculations under one single output file and after some research and some talks with a few other students, I found out that it was possible to context.write() many times and that there existed a method called cleanup which is called after all the map tasks are completed.

Week 5: During this week, I managed to calculate the minimum and maximum values for the variable "massExcessUncertainty" from the dataset "NuclearDecay.csv". I decided that I was going to calculate all variables separately in different java files and congregate everything at the end.

Week 6: During this week, I managed to calculate the median and mode for the NuclearDecay.csv dataset separately (in different java files). However, at the end of this week I tried to aggregate all the separate variable calculations under one single java file, which was successful to a certain extent.

Week 7: During this week, I managed to finally implement the mode and median calculations into NuclearDecay.java. Thus, I managed to get the java file to output all variables under a single file. In the end, I deleted any unnecessary java files.

Week 8: During this week, I wrote a java file that calculated the cumulative frequencies for each of the values in the "massExcessUncertainty" column and I used python to generate a graph based on those values. Its important to note that I reused the code that I wrote for the partial calculation of the mode for problem 3 and I borrowed a code snippets regarding the calculation of the interval and length of the bars as I was not too familiar with matplotlib (only using it a few times for my machine learning module)

Week 9: During this week, I managed to calculate the daily price range and change for a certain stock from a NASDAQ dataset. This calculation did not prove to be difficult at all since it was very basic. Moreover, I generated a graph for the calculation of the price range/change using python.

Week 10: During this week, I managed to calculate the volume rate of change for the same stock and generated a graph using python in order to visualise the daily volume rate of change for that stock. The calculations were not too complicated as the equation for this calculation was basic. Moreover, during this week I started to work on my report.

Week 11: By this week, most of the code was finished as I managed to calculate the final metric

for problem 4 which was the RSI. This calculation proved to be a bit more complicated than the others as the equation can be confusing, nevertheless, after some research it did not prove to be too difficult. Moreover, By this week I was nearly finished with the Rationale, the Professional Issues and the Literature Review sections of my code. However, my final draft to my supervisor was somewhat incomplete and the feedback I received was limited.

Week 12: During this week, I mainly worked on the remaining sections of my report. However, I also made slight changes to my shell script and to my RegexSearch.java file such that it is now capable of taking a regex expression as input directly from the command line.

8.2 README file

The video for how to run the code can be found on Gitlab or in the submission folder

When running WordCount use the following commands:

```
ssh bigdata
cd /home/cim/ug/zkac422/PROJECT/
javac WordCount.java -cp $(hadoop classpath)
jar cvf wordcount.jar *.class
hadoop jar wordcount.jar WordCount poems WordCountOutput1
```

//remember that WordCountOutput1 is the name of the folder and that if your folder is already stored in the hadoop file system, then you will get an error, in order to overcome this issue, you may want to call your folder a different name or you may want to delete the folder from the hadoop system using "hadoop fs -rm -r *nameoffolder*. Moreover, you can check which folders/files are contained in the hadoop file system by using the command hadoop fs -ls

```
hadoop fs -get WordCountOutput1
```

//note that if the folder is already on the local system, you will get an error. In order to avoid this issue, you may want to eliminate the already existing folder using rm -r *nameoffolder* or by just directly deleting it off from the file system cd WordCountOutput

```
cat part-r-00000
```

//use cd .. to go back to the "PROJECT" folder

When running RegexSearch use the following commands:

```
javac RegexSearch.java LineNumberInputFormat.java -cp $(hadoop classpath)
jar cvf regexsearch.jar lineinputformat.jar *.class
hadoop fs -rm -r RegexSearchOutput
rm -r RegexSearchOutput
hadoop jar regexsearch.jar RegexSearch philosophy RegexSearchOutput ("h\\w+")
hadoop fs -get RegexSearchOutput
cd RegexSearchOutput
cat part-r-00000
```

When running a python file, for example, trading.py, use the following commands:

```
rm -r tradingrange.png
python tradingrange.py
```

It is important to note that if you have any doubts regarding how to run a python file, just check the Test.sh script and run any of the commands present in there.

Moreover, just using the Test script would make it easier to check the results of each file

9 Acronyms

NASDAQ: National Association of Securities Dealers Automated Quotations

WWW: World Wide Web

IT: Information Technology

HDFS: Hadoop Distributed File System

YARN: Yet Another Resource Negotiator

K-means: A clustering algorithm

CAGR: Compound Annual Growth Rate

LMS: Learning Management Systems

IoT: Internet of Things

AWS: Amazon Web Services

GCP: Google Cloud Platform

Azure: Microsoft Azure

FYP: Final Year Project

TDD: Test Driven Development

JVM: Java Virtual Machine

CPU: Central Processing Unit

GC: Garbage Collection

RSI: Relative Strength Indicator

VROC: Volume Rate of Change

PC: Price Change

TR: Trading Range

10 Glossary

MapReduce: A programming model and processing technique for parallel and distributed computing.

Apache Hadoop: An open-source framework for distributed storage and processing of big datasets.

Hadoop Distributed File System (HDFS): A distributed file system designed to store vast amounts of data reliably across a cluster of machines. It is part of the Hadoop ecosystem.

Yet Another Resource Negotiator (YARN): A resource management layer in Hadoop responsible for managing and monitoring cluster nodes and resource utilization. It is part of the Hadoop ecosystem.

Hadoop Common: A set of shared utilities and libraries used across all Hadoop modules. It is part of the Hadoop ecosystem.

Apache Spark: An open-source, distributed computing system that provides in-memory processing for big data.

Apache Superset: An open-source data exploration and visualisation platform.

Apache Flink: An open-source stream processing framework for big data processing and data analytics.

Amazon EMR: A cloud based big data platform that simplifies large scale data processing.

Decomposition: Breaking down complex problems into smaller, manageable tasks.

DRY (Don't Repeat Yourself): A software development principle emphasizing code reusability and avoiding redundancy.

KISS (Keep It Simple, Stupid): A design principle promoting simplicity and clarity in code and systems.

Git: A distributed version control system for tracking changes in source code during software development.

ByteOffset: The position of a byte within a file.

Software Engineering: The application of engineering principles to software development

Abstraction: The process of hiding the implementation details of a systems components

Decomposition: Breaking down a complex problem into smaller components.

Encapsulation: The bundling of data and methods that operate on the data into a single unit

Inheritance: A mechanism in object-oriented programming where a class inherits properties and behaviors from another class

Exception Handling: The process of managing errors during the execution of a program

11 Bibliography

- [1] “Mapreduce tutorial,” (tutorial where I got myself started with Hadoop). [Online]. Available: <https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>
- [2] T. White, *Hadoop: The Definitive Guide*. Sebastopol, CA: O’Reilly Media, Inc., 2009, (An introduction to basic data analytics with Hadoop).
- [3] “What is hadoop,” (History about the Hadoop project). [Online]. Available: <https://www.javatpoint.com/what-is-hadoop>
- [4] “Healthcare big data projects, applications and examples,” (Hadoop applications in healthcare). [Online]. Available: <https://www.projectpro.io/article/5-healthcare-applications-of-hadoop-and-big-data/85>
- [5] A. O’Driscoll, J. Daugelaite, and R. D. Sleator, “‘big data’, hadoop and cloud computing in genomics,” *Journal of Biomedical Informatics*, vol. 46, no. 5, pp. 774–781, 2013, (Hadoop maps human genome). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1532046413001007>
- [6] “The impact of hadoop and big data in finance and banking industries,” (Hadoop’s impact on the financial sector). [Online]. Available: <https://www.chetu.com/blogs/finance-2/hadoop-and-big-data-in-finance-and-banking.php>
- [7] “The nsa and big data?” 2013, (NSA’s involvement in Bigdata). [Online]. Available: <https://www.informationweek.com/machine-learning-ai/the-nsa-and-big-data-what-it-can-learn>
- [8] “Using big data to bridge the virtual physical worlds,” (Nokia’s usage of Hadoop). [Online]. Available: https://hadoopilluminated.com/hadoop_illuminated/cached_reports/Cloudera_Nokia_Case_Study_Hadoop.pdf
- [9] “Mass excess,” (What is mass excess). [Online]. Available: https://www.chemeurope.com/en/encyclopedia/Mass_excess.html/
- [10] “Relative strength index (rsi) indicator,” (What is rsi). [Online]. Available: <https://www.investopedia.com/terms/r/rsi.asp/>
- [11] “Price change: Definition, types, causes effects,” (What is price change). [Online]. Available: <https://www.investopedia.com/terms/p/price-change.asp/>
- [12] “Trading range: Definition, when it occurs, how to use and example.” [Online]. Available: <https://www.investopedia.com/terms/t/tradingrange.aspnote={Whatistradingrange}>
- [13] “Volume rate of change,” (What is vroc). [Online]. Available: <https://trendspider.com/learning-center/volume-rate-of-change/>
- [14] “Basic map and reduce algorithm design,” (Big data ethics). [Online]. Available: https://www.dcs.bbk.ac.uk/~dell/teaching/cc/book/ditp/ditp_ch3.pdf/
- [15] “What is garbage collection? automated memory management for your programs,” (Garbage Collection). [Online]. Available: <https://www.infoworld.com/article/3685493/what-is-garbage-collection-automated-memory-management-for-your-programs.html/>

- [16] “Sample custom inputformat class for hadoop,” 2015, (Github to the code that I borrowed and used to modify TextInputFormat such that it outputs the line number instead of the ByteOffset, this is also the primary resource I relied upon to change the code). [Online]. Available: <https://gist.github.com/dedunumax/96594b09d0f566e88ced>
- [17] “Recordreader,” (Web page I used to understand how RecordReader works). [Online]. Available: <https://waytoeasylearn.com/learn/hadoop-recordreader/>
- [18] “Hadoop recordreader – how recordreader works in hadoop?” (Second web page I used to understand how RecordReader works). [Online]. Available: <https://data-flair.training/blogs/hadoop-recordreader/>
- [19] “how to set custom input format in mapreduce?” 2013, (StackExchange discussion where I borrowed bits of the code and utilised to modify FileInputNumberFormat). [Online]. Available: <https://stackoverflow.com/questions/15598537/how-to-set-custom-input-format-in-mapreduce>
- [20] “Custom n line record reader in hadoop,” (A resource I utilised in order to modify the code for FileInputNumberFormat). [Online]. Available: <https://www.kamalsblog.com/2017/07/custom-n-line-record-reader-in-hadoop.html?m=1>
- [21] “Frequency distribution calculation,” (A resource I utilised in order to calculate the frequency table FrequencyDistribution.py). [Online]. Available: <https://stackoverflow.com/questions/41551658/how-to-create-a-frequency-distribution-table-on-given-data-with-python-in-jupyter>
- [22] “Bar interval calculation,” (A resource I utilised in order to calculate the bar interval for FrequencyDistribution.py). [Online]. Available: <https://www.geeksforgeeks.org/how-to-create-frequency-tables-in-python>
- [23] M. Wang, W. Huang, F. Kondev, G. Audi, and S. Naimi, “The ame 2020 atomic mass evaluation (ii). tables, graphs and references*,” *Chinese Physics C*, vol. 45, no. 3, p. 030003, mar 2021. [Online]. Available: <https://dx.doi.org/10.1088/1674-1137/abddaf>
- [24] “Agilent company history,” (Agilent company history). [Online]. Available: https://www.agilent.com/about/companyinfo/history/timeline_2010s.html/
- [25] “2007 agilent technologies inc. annual report,” (annual report by Agilent). [Online]. Available: https://www.annualreports.com/HostedData/AnnualReportArchive/a/NYSE_A.2007.pdf/
- [26] “A tale of three crises in the past two decades,” (Financial crisis in the stock market). [Online]. Available: https://indexes.nasdaqomx.com/docs/Nasdaq-100_A%20Tale%20of%20Three%20Crises%20over%20Two%20Decades.pdf/
- [27] “Histogram midpoint calculation,” (A resource I utilised in order to calculate the midpoint for FrequencyDistribution.py). [Online]. Available: <https://stackoverflow.com/questions/40825208/python-histogram-how-to-find-the-midpoint-of-bin-with-the-maximum-frequency/>
- [28] “Ethical considerations in data analytics: Balancing insights with privacy,” (ethical consideration in data analytics). [Online]. Available: <https://skillfloor.medium.com/ethical-considerations-in-data-analytics-balancing-insights-with-privacy-3b70e2b779c9/>
- [29] “The right to be forgotten,” (why storing data long term is unethical). [Online]. Available: https://repository.uclawsf.edu/hastings.law_journal/vol64/iss2/6/
- [30] “Howe iii, e. g., elenberg, f. (2020). ethical challenges posed by big data. innovations in clinical neuroscience, 17(10-12), 24–30.” (Big data ethics). [Online]. Available: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7819582/>